BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE ENGLISH

DIPLOMA THESIS

Development of a Context-Aware Mobile Application for Personalized Fitness and Nutrition

Supervisor
Associate Professor, PhD. ZSIGMOND Imre

Author
Bolchis Razvan

2026

ABSTRACT

This thesis presents FitTrack, a cross-platform mobile application for personalized fitness and nutrition coaching, developed with React Native and TypeScript. The system combines workout planning, nutrition tracking, and social motivation features in a single user-centered architecture. Unlike traditional fitness applications that treat these domains separately, FitTrack integrates them into a unified feedback loop that supports both short-term consistency and long-term habit formation.

The theoretical part of this thesis focuses on several areas of mobile health (mHealth), including recommendation systems, personalized nutrition, user-centered design, and the impact of social interaction on user motivation. Based on these ideas, FitTrack was developed using Firebase services for authentication and data storage. This ensures secure account access and allows user progress to be stored and shared across the application's modules.

In terms of functionality, FitTrack allows users to follow personalized workout recommendations, track their meals with detailed macronutrient information, and use predefined meal plans designed for different fitness goals. The nutrition module combines a local food database with external food search services, providing a wider range of food options and making daily tracking more accurate. The application also includes community features such as challenges, feed sharing, leaderboards, and achievement badges, which help users stay motivated and engaged with their fitness goals.

One of the main contributions of FitTrack is the integration of personalized adaptation based on user activity. Recommendations and fitness targets can be updated according to user progress, workout history, and available sensor data, including step tracking when supported by the device and platform permissions. In addition, the application uses Firebase synchronization and state management mechanisms to keep user data consistent and up to date across its modules.

Overall, FitTrack shows how fitness tracking, nutrition management, and social features can be combined within a single mobile application. By bringing these components together, the platform aims to help users stay motivated, follow their goals more consistently, and maintain a better overview of their health and fitness progress.

Bolchis Razvan

Author's Declaration: This thesis represents original work completed independently. In its preparation, no unauthorized collaboration or assistance was involved.

Contents

1 Introduction **1**

1.1 Mobile Fitness and Health Applications 1
1.2 Importance of Personalized Nutrition and Exercise 1
1.3 Project Mission and Objectives 3
1.4 Cross-Platform Mobile Development Overview 4
1.5 Thesis Structure and Contributions 5

2 Technical Implementation and Architecture **6**

2.1 Technology Stack and Development Environment 6
2.1.1 React Native Framework 6
2.1.2 Firebase Backend Services 7
2.1.3 TypeScript and Development Tooling 7
2.2 Application Architecture 8
2.2.1 Component Structure and Organization 8
2.2.2 Navigation System Implementation 9
2.2.3 State Management with Context API 9
2.3 Core Modules Implementation 10
2.3.1 Nutrition Module 10
2.3.2 Workout Module 12
2.3.3 Dashboard Module 16
2.3.4 Profile Module 18
2.3.5 Community Module 19
2.4 Database Design and Data Models 21
2.5 Authentication and Security Implementation 22
2.6 Testing and Validation 23
2.7 Implementation Status and Milestones 24
2.8 Limitations and Future Work 25
2.9 Chapter Summary 27

3 User Interface Design and Implementation **28**

3.1 Design Philosophy and Principles 28

3.1.1 Core Design Principles 29
3.1.2 Visual Hierarchy and Information Architecture 29
3.2 Screen-Level Design and Implementation 30
3.2.1 Dashboard Screen 30
3.2.2 Nutrition Screen 31
3.2.3 Workout Screen 33
3.2.4 Profile Screen 35
3.2.5 Community Screen 36
3.3 Navigation Implementation 38
3.3.1 Navigation Patterns and Hierarchies 38
3.3.2 State Preservation and Deep Linking 39
3.4 Component Design, Layout, and Responsiveness 39
3.4.1 Core Reusable Components 39
3.4.2 Layout Patterns and Responsive Design 40
3.5 Theme System and Dark Mode 40
3.5.1 Theme Architecture 40
3.5.2 Theme Switching and Persistence 40
3.6 Visual Feedback and Animation 41
3.6.1 Interaction Feedback 41
3.6.2 State Transition Animations 41
3.7 Accessibility and Inclusive Design 42
3.7.1 Screen Reader Support 42
3.7.2 Motor Accessibility 42
3.7.3 Cognitive Accessibility 42
3.8 Chapter Summary 43

4 Implementation Results and Evaluation **44**
4.1 Implementation Status and Feature Coverage 44
4.2 Testing and Validation 47
4.3 Performance Evaluation 47
4.4 Key Challenges and Solutions 49
4.5 Current Limitations and Future Enhancements 50
4.6 Chapter Summary 51

5 Conclusions and Future Work **52**
5.1 Project Summary and Achievements 52
5.2 Insights and Lessons Learned 53
5.3 Future Development Directions 54
5.4 Reflections on Mobile Health Development 55

6 Conclusions **56**
6.1 Fulfillment of Objectives 56
6.2 Main Contributions 57
6.3 Final Conclusions 57

Bibliography **59**

Chapter 1

Introduction

1.1 Mobile Fitness and Health Applications

Fitness and wellness have become increasingly important aspects of everyday life due to their impact on both physical and mental health. As interest in preventive healthcare continues to grow, many people rely on digital tools to help them maintain healthy habits and track their progress over time. Among these tools, mobile health applications, commonly referred to as *mHealth*, have gained significant popularity because they are easy to access, available at any time, and can support users throughout their daily activities.

The widespread use of smartphones and wearable devices has significantly increased the capabilities of mobile health applications. Today, these applications can track physical activity, monitor nutrition, organize workout plans, provide progress feedback, and support interaction between users with similar fitness goals[ZDP+14, PAV15]. Despite these advances, many fitness applications still rely on general recommendations and predefined plans instead of adapting to individual user needs, progress, and everyday constraints such as available time or equipment. Recent research has shown that context-aware and personalized interventions can improve user engagement and adherence to physical activity goals [LGKM20].

1.2 Importance of Personalized Nutrition and Exercise

Research in public health has consistently shown that regular physical activity and healthy eating habits can reduce the risk of chronic diseases while improving both physical and mental well-being. Regular exercise has also been associated with positive effects on mental health and reduced symptoms of depression [SVS+17]. However, these benefits are often greater when recommendations are tailored to the individual rather than based on a general approach.

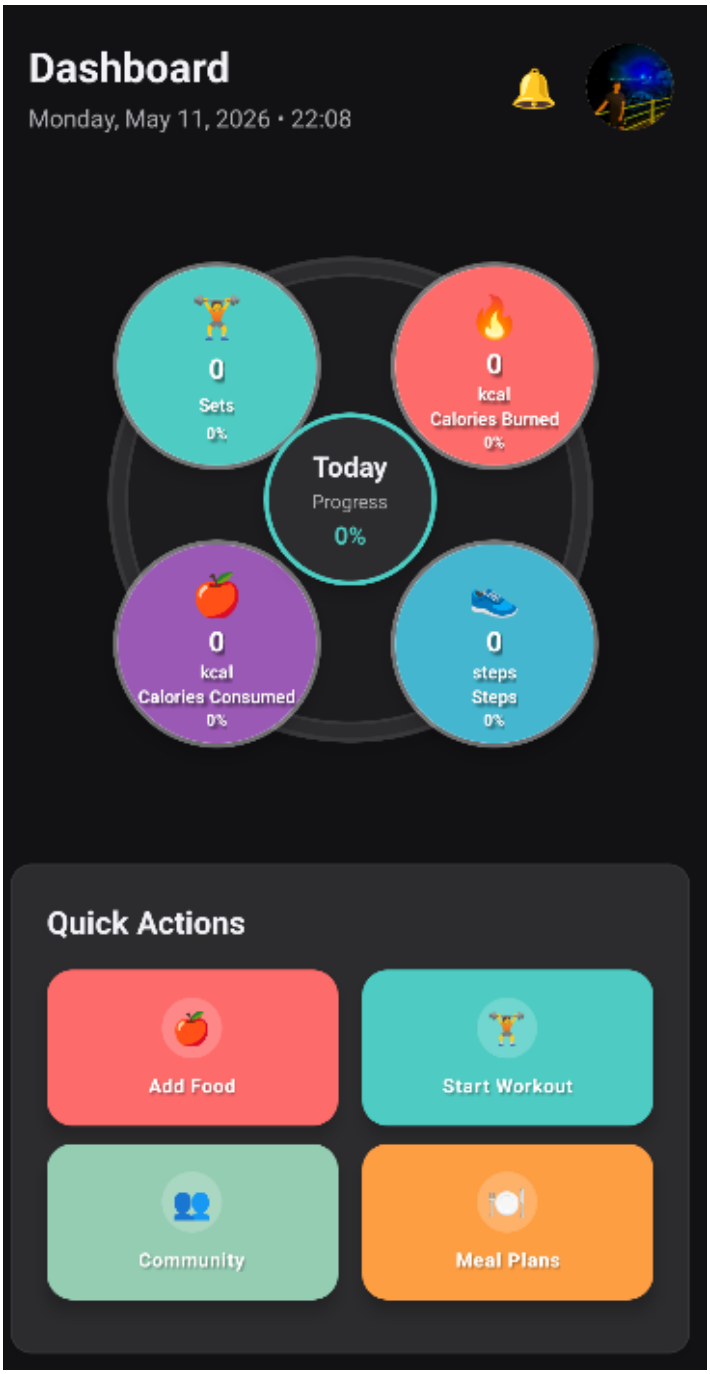

Figure 1.1: FitTrack dashboard overview showing daily progress indicators and quick-access actions.

In mobile fitness applications, personalization means adapting recommendations to the user's characteristics, goals, and activity history. Factors such as fitness level, training objectives, workout frequency, and nutrition habits can influence the type of guidance provided by the application. Similar approaches have been explored in both physical activity coaching systems and personalized nutrition recommendation platforms, where user-specific data is used to generate more relevant suggestions and improve long-term adherence [LGKM20, YHY+17, PAL20].

FitTrack follows this approach by combining workout planning, exercise tracking, nutrition management, and personalized meal templates within a single platform. By bringing these features together, the application helps users monitor both their training progress and dietary habits in one place.

1.3 Project Mission and Objectives

This thesis presents the design and implementation of *FitTrack*, a mobile application that combines fitness tracking, nutrition management, and social features within a single platform. The main objectives of the project are:

- To develop an intuitive mobile interface that allows users to easily navigate between the dashboard, nutrition, workout, meal planning, profile, and community sections.
- To implement nutrition features for meal logging, macronutrient tracking, food search, filtering and sorting options, as well as usability improvements such as pagination and inline editing.
- To provide workout management tools that support exercise discovery through an interactive body map, integration with external exercise databases, personal routine creation, workout session tracking, weekly planning, and progress analysis.
- To provide detailed exercise information while maintaining good application performance by loading instructional media only when users open the exercise details screen.
- To support personalized user profiles with editable personal information, preferences, and data export functionality.
- To implement community features such as fitness challenges, challenge sharing, leaderboards, and achievement badges that encourage user participation and motivation, as gamification mechanisms have been shown to positively influence engagement and adherence in health-related applications [HKS14].

- To integrate Firebase services for authentication and data synchronization, ensuring that user information, workout routines, and progress data remain available and consistent across the application.

1.4 Cross-Platform Mobile Development Overview

Supporting both Android and iOS was one of the main reasons for choosing a cross-platform development approach. FitTrack was developed using **React Native** and **TypeScript**, allowing most of the codebase to be shared across both platforms while maintaining a native user experience. The backend is powered by **Firebase**, which provides authentication and cloud data storage through Firestore, enabling users to securely access and synchronize their information across the application.

The workout module also integrates external exercise catalogs, allowing users to access a large collection of exercises without maintaining all data locally. Exercise information is retrieved and adapted to the application's data structure, while optional filters such as muscle groups help users find relevant exercises more easily. Detailed instructions and demonstration videos are loaded only when an exercise is opened, reducing unnecessary data usage and helping maintain smooth performance on mobile devices.

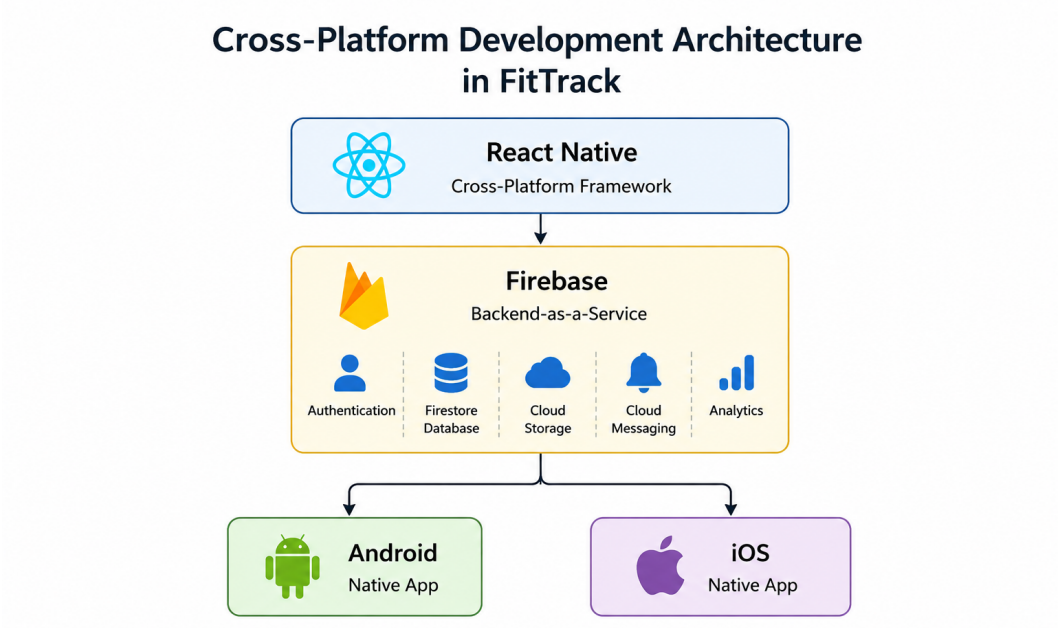

Figure 1: Cross-platform architecture using React Native and Firebase

Figure 1.2: Cross-platform architecture of FitTrack. React Native provides a shared codebase for Android and iOS, while Firebase delivers authentication, cloud storage, notifications, analytics, and database services.

1.5 Thesis Structure and Contributions

The remainder of this thesis is organized as follows:

- **Chapter 2** presents the technical implementation of the application, including the technology stack, system architecture, core modules (nutrition, workout, dashboard, profile, and community), database design, authentication, data synchronization, testing, and implementation challenges.
- **Chapter 3** focuses on the design and implementation of the user interface, covering navigation, visual hierarchy, theming, responsiveness, and accessibility considerations.
- **Chapter 4** presents the implementation results and evaluation of the application, including feature coverage, testing results, performance aspects, challenges encountered during development, and current limitations.
- **Chapter 5** discusses the main lessons learned during development and outlines possible future improvements.
- **Chapter 6** summarizes the main contributions and conclusions of this thesis.

Contribution emphasis: The main contribution of FitTrack is the integration of fitness tracking, nutrition management, and community features within a single mobile application. Instead of treating these areas as separate components, the platform brings together workout planning, exercise tracking, meal logging, personalized meal plans, and social challenges in a unified environment. This approach allows users to manage different aspects of their fitness journey through a single application while maintaining consistent access to their data and progress.

Chapter 2

Technical Implementation and Architecture

This chapter describes the technical implementation of the FitTrack mobile application, including the technologies used, the overall system architecture, and the main application modules. It also explains the design decisions made during development and how the different components work together to support the application's functionality.

2.1 Technology Stack and Development Environment

FitTrack was developed using a technology stack designed to support both Android and iOS from a single codebase. The selected technologies provide good performance, easier maintenance, and a more efficient development process. In addition, the use of TypeScript helps reduce common programming errors through static type checking, improving code reliability as the application grows.

2.1.1 React Native Framework

React Native (v0.79.1) serves as the main framework used in the development of FitTrack. It allows the application to run on both Android and iOS while sharing most of the codebase between platforms. At the same time, React Native provides access to native device APIs, which are required for features such as animations, sensor integration, and real-time interface updates. These capabilities are particularly important in a fitness application, where users expect smooth interactions and immediate feedback during everyday use.

Key advantages in FitTrack's context:

- *Cross-platform consistency* through a unified codebase and shared UI logic.

- *Native performance* with direct access to native components and APIs.
- *Rapid iteration* via hot reloading and efficient debugging workflows.
- *TypeScript support* for compile-time guarantees and safe refactoring.

2.1.2 Firebase Backend Services

Firebase was selected as the backend solution for FitTrack because it provides authentication, cloud data storage, and file management within a single platform. The Firebase configuration is centralized in `src/config/firebase.ts`, making initialization and maintenance easier throughout the application. In addition, Firestore's real-time synchronization allows user data, such as nutrition logs, workout sessions, and progress information, to remain up to date across the application's modules. Real-time cloud synchronization is a common requirement in modern mobile health applications, where data consistency across multiple features directly affects user experience and engagement [SGL+21].

Service	**Core Functionality**	**Role in FitTrack**
Firebase Authentication	OAuth 2.0 (Google), email/password, anonymous	Secure sign-in; session handling with token refresh
Cloud Firestore	NoSQL, real-time sync, offline support	User-scoped routines, sessions, nutrition logs, community data
Firebase Storage	Media/object storage	Profile images and application assets
Firebase Console	Monitoring and configuration	Operational visibility and maintainability

Table 2.1: Firebase services and their roles within FitTrack

2.1.3 TypeScript and Development Tooling

TypeScript is used throughout the application to provide static type checking and improve code reliability. By detecting many errors during development rather than at runtime, it helps reduce common programming mistakes and makes the code easier to maintain. TypeScript also improves IDE support through features such as autocompletion and type hints, making development and refactoring more efficient.

Aspect	TypeScript	JavaScript
Type Safety	Detects many errors during development through static type checking.	Errors are usually discovered at runtime.
Refactoring	Safer to refactor because IDEs can identify type-related issues.	Refactoring can introduce unnoticed errors.
Documentation	Types and interfaces make the code easier to understand.	Code understanding relies more on comments and external documentation.
Maintainability	Easier to maintain as the codebase grows.	Can become harder to manage in larger projects.

Table 2.2: Comparison between TypeScript and JavaScript

Development tools used:

- *ESLint* for code quality checks and coding standards.
- *Prettier* for automatic code formatting.
- *Jest* for unit testing.
- *React Native CLI* for building and running the application.
- *Metro* as the JavaScript bundler used by React Native.

2.2 Application Architecture

The application is organized using a modular architecture, where each module is responsible for a specific area of functionality. This approach helps keep the codebase easier to understand, maintain, and extend as new features are added. Shared data is managed through React Context, allowing different parts of the application to communicate while keeping responsibilities clearly separated.

2.2.1 Component Structure and Organization

The application is divided into six main modules that provide its core functionality.

The *authentication* module handles user registration, login, and session management while protecting access to user-specific features.

The *nutrition* module allows users to search for foods, log meals, and monitor their daily macronutrient intake.

The *workout* module provides exercise discovery, personal routine management, workout tracking, and weekly planning features.

The *dashboard* module offers an overview of key fitness and nutrition information, including calories, steps, active minutes, and workout progress.

The *profile* module stores personal information, user preferences, and data export options.

The *community* module includes challenges, leaderboards, social interactions, and notifications that encourage users to stay active and engaged.

2.2.2 Navigation System Implementation

Navigation within FitTrack is implemented using `React Navigation`. Different navigation types are used depending on the purpose of each screen, helping users move easily through the application while restricting access to features that require authentication.

Navigation types used:

- Stack navigation for navigation between related screens.
- Tab navigation for the main application sections (Dashboard, Nutrition, and Workout).
- Modal navigation for temporary screens such as quick actions and exercise details.
- Authentication guards to restrict access based on user login status.

2.2.3 State Management with Context API

React Context API is used to manage shared application data without passing information through multiple component levels. Four main contexts are defined in `App.tsx`: *ThemeContext*, *NutritionContext*, *WorkoutContext*, and *NotificationInboxContext*. These contexts provide access to theme settings, nutrition data, workout information, and notifications throughout the application. Where required, the data is synchronized with Firestore so that user progress and preferences remain available across sessions.

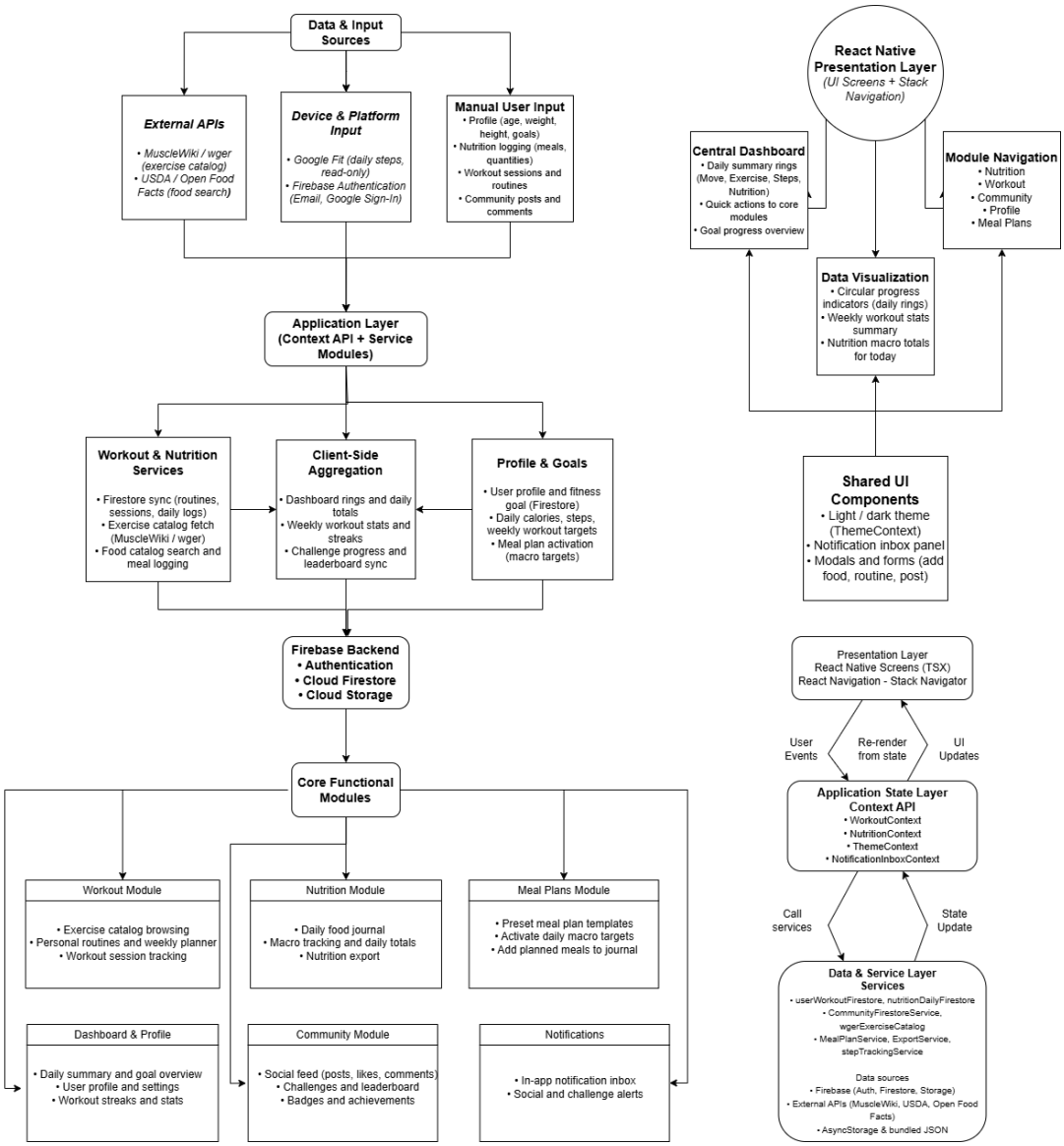

Figure 2.1: High-level system architecture showing data flow from input sources to core modules.

2.3 Core Modules Implementation

2.3.1 Nutrition Module

The nutrition module allows users to search for foods, log meals, and monitor their daily calorie and macronutrient intake. Personalized nutrition support has become an important area of research within digital health systems, with recommendation-based approaches showing potential for improving dietary adherence and long-term engagement [YHY+17, PAL20]. A search interface is provided together with filtering and sorting options, including calorie-based sorting, macronutrient prioritization, and dietary preferences such as vegan or vegetarian foods. Food items

are displayed using `FlatList` components, and selecting an item opens a detailed view where users can adjust serving size and automatically recalculate nutritional values.

Meal entries are stored separately for each user and organized by meal categories such as breakfast, lunch, and dinner. The module calculates daily totals for calories and macronutrients and compares them with a user-defined nutrition goal, which is set to 2000 kcal by default. A progress bar provides a visual overview of daily intake, while nutrition summaries are also displayed on the dashboard. Data is managed through *NutritionContext* and synchronized with Firestore, allowing users to save, update, or remove individual meal entries. The module also includes meal templates and export functionality for users who want to plan or review their nutrition over longer periods.

Meal Plans and Nutrition Templates

In addition to daily meal logging, the nutrition module includes predefined meal plans designed for different goals, such as fat loss, maintenance, and muscle gain. Each meal plan contains a full-day structure with meals, ingredients, calories, and macronutrient values.

Users can browse available templates, view detailed meal plan information, and activate a selected plan for the current day. This feature helps users plan their nutrition more easily and provides a practical starting point for consistent meal tracking. Similar approaches have been explored in personalized meal recommendation systems that adapt nutritional suggestions to user preferences and goals [YHY+17].

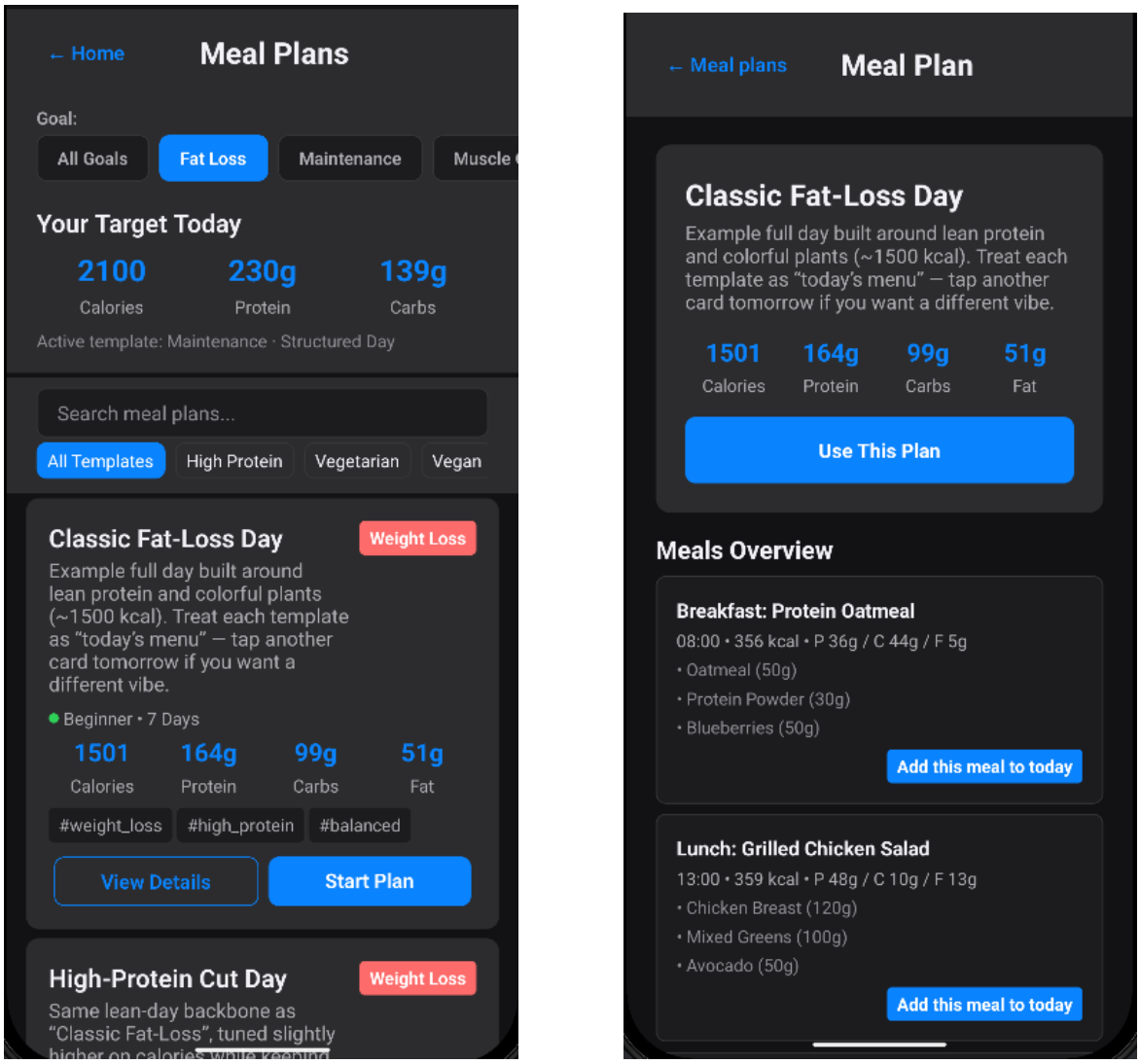

Figure 2.2: Meal plan templates and detailed meal-plan view with nutritional targets.

2.3.2 Workout Module

The workout module combines exercise discovery, workout planning, and workout tracking within a single interface. Users can browse exercises, create personal routines, follow guided workout sessions, and organize their training schedule through a weekly planner. Exercise data is retrieved from external catalogs and adapted to a format that can be used consistently throughout the application. Such features are commonly found in modern fitness applications, where exercise tracking and activity monitoring are often combined with wearable-device ecosystems and personalized recommendations [JZ16, PAV15].

External Exercise Catalog Integration

The main source of exercise data is the official MuscleWiki API, configured through a local credentials file (`muscleWiki.local.ts`) that is excluded from version control. If the API key is unavailable, the application can fall back to the public wger.de catalog or to a locally bundled dataset used for testing and demonstration purposes.

Users can filter exercises through an interactive body map, which allows muscle groups to be selected using both front and back body views. Additional filters are available for difficulty level and required equipment, making it easier to find exercises that match specific training goals.

To maintain good performance, exercise lists load only the information required for browsing. Detailed instructions and demonstration videos are retrieved only when a user opens a specific exercise, reducing unnecessary data usage and keeping the interface responsive even when large exercise catalogs are available.

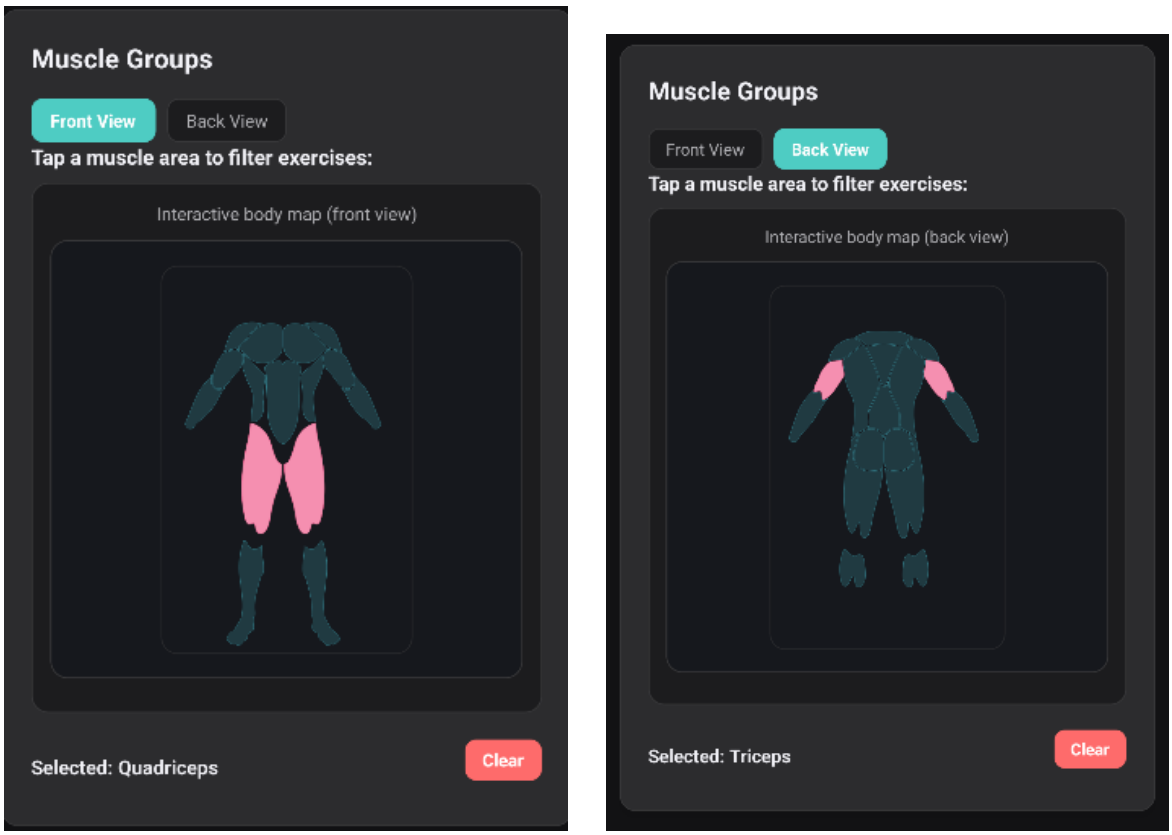

Figure 2.3: Interactive body map used to filter exercises by muscle group (front and back views).

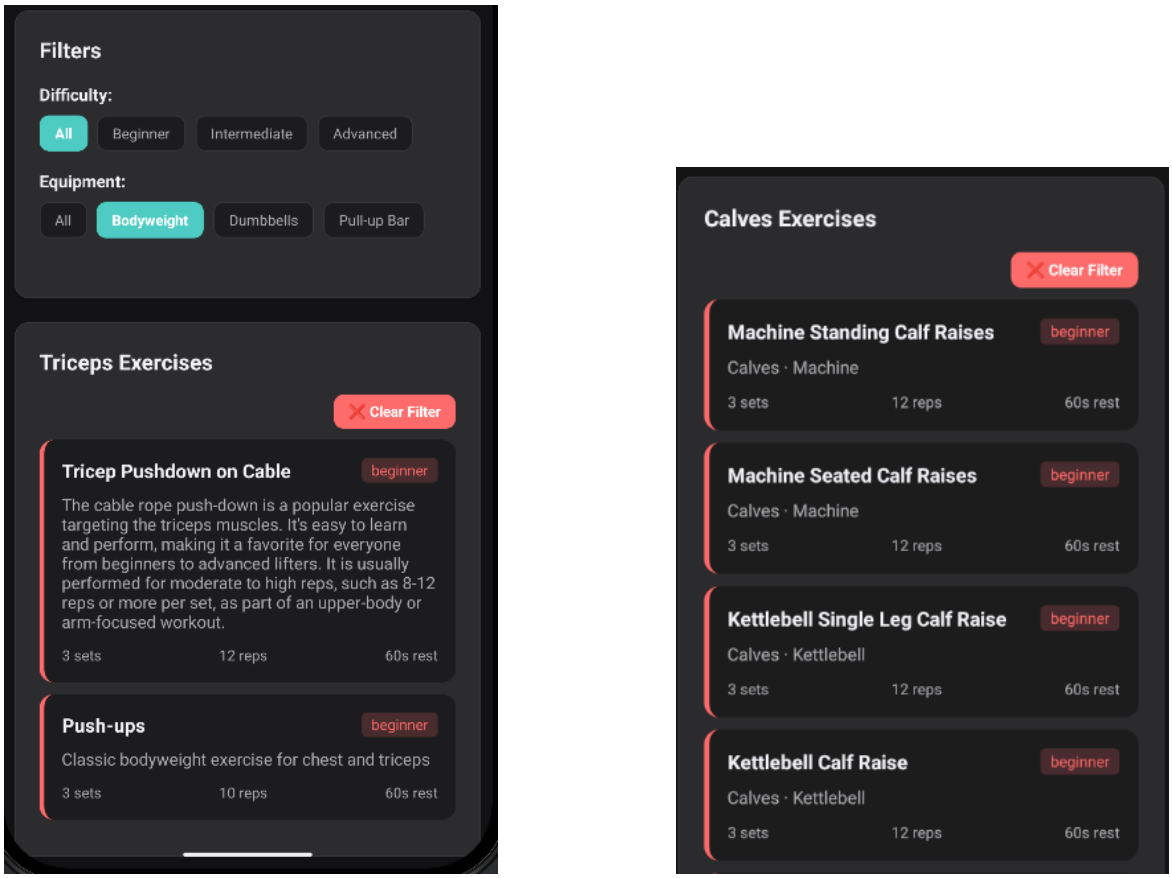

Figure 2.4: Exercise filtering interface and resulting exercise catalog.

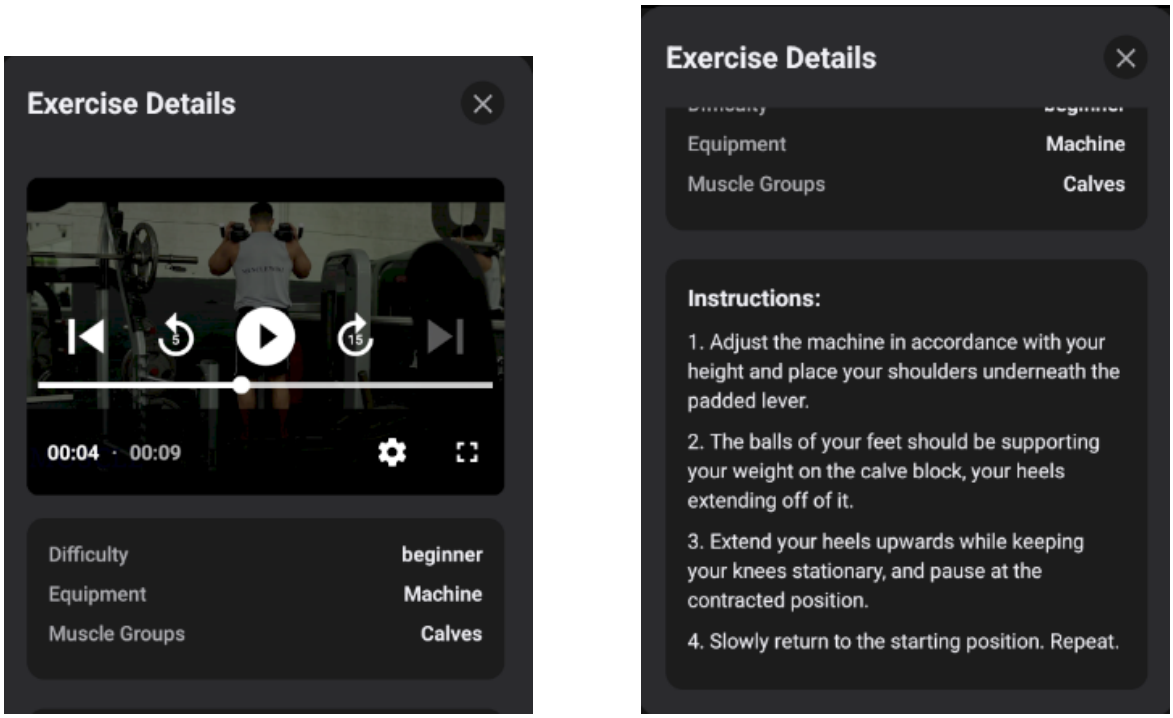

Figure 2.5: Exercise detail view showing demonstration media, exercise information, and step-by-step instructions.

Personal Routines and Workout Tracking

Authenticated users can create and manage their own workout routines, which are stored in Firestore under `users/uid/routines`. Each routine contains the selected exercises together with training parameters such as sets, repetitions, rest periods, and exercise order.

Exercises can be added directly from the exercise details screen, where users may either select an existing routine or create a new one. This simplifies routine creation and allows newly discovered exercises to be quickly incorporated into personalized training plans.

During a workout session, the application tracks completed sets, repetitions, and rest periods using integrated countdown timers. Once a session is completed, the results are saved to Firestore and become available for progress tracking and dashboard statistics. Users can also organize their training schedule through a weekly planner, assigning workout routines to specific days of the week.

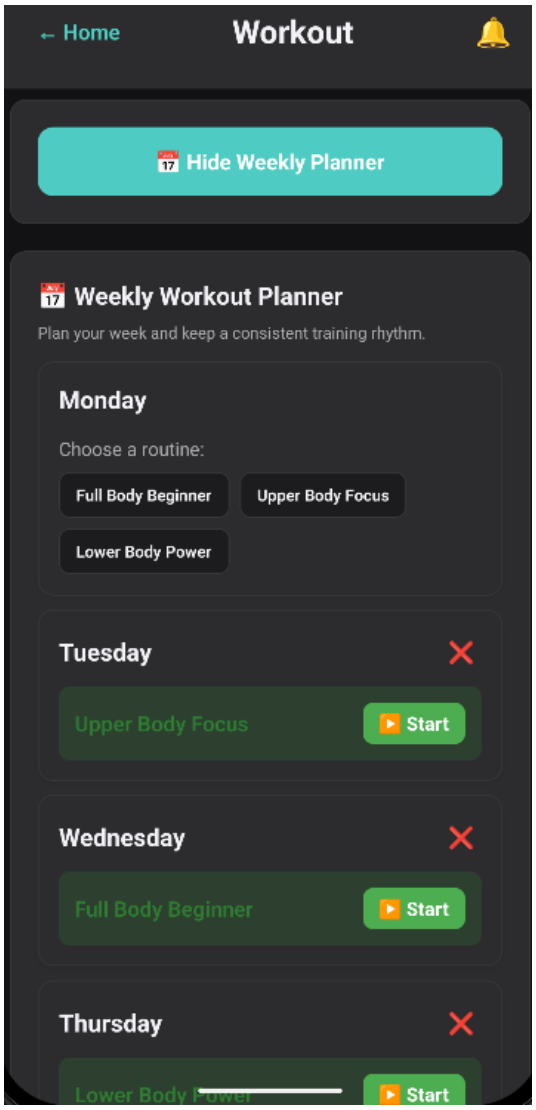

Figure 2.6: Weekly workout planner used to assign routines to specific days of the week.

2.3.3 Dashboard Module

The dashboard provides a centralized overview of the user's daily fitness and nutrition progress. Centralized feedback dashboards are frequently used in digital health systems because they improve awareness of personal progress and support self-monitoring behaviors associated with healthier lifestyles [SHKW15]. A circular visualization displays key indicators such as movement, exercise activity, step count, and nutrition progress, allowing users to quickly assess their current status. Additional progress bars provide more detailed information while keeping the interface clean and easy to read.

The dashboard also includes quick-access actions such as adding meals, starting workouts, and accessing frequently used modules. Data displayed on the dash-

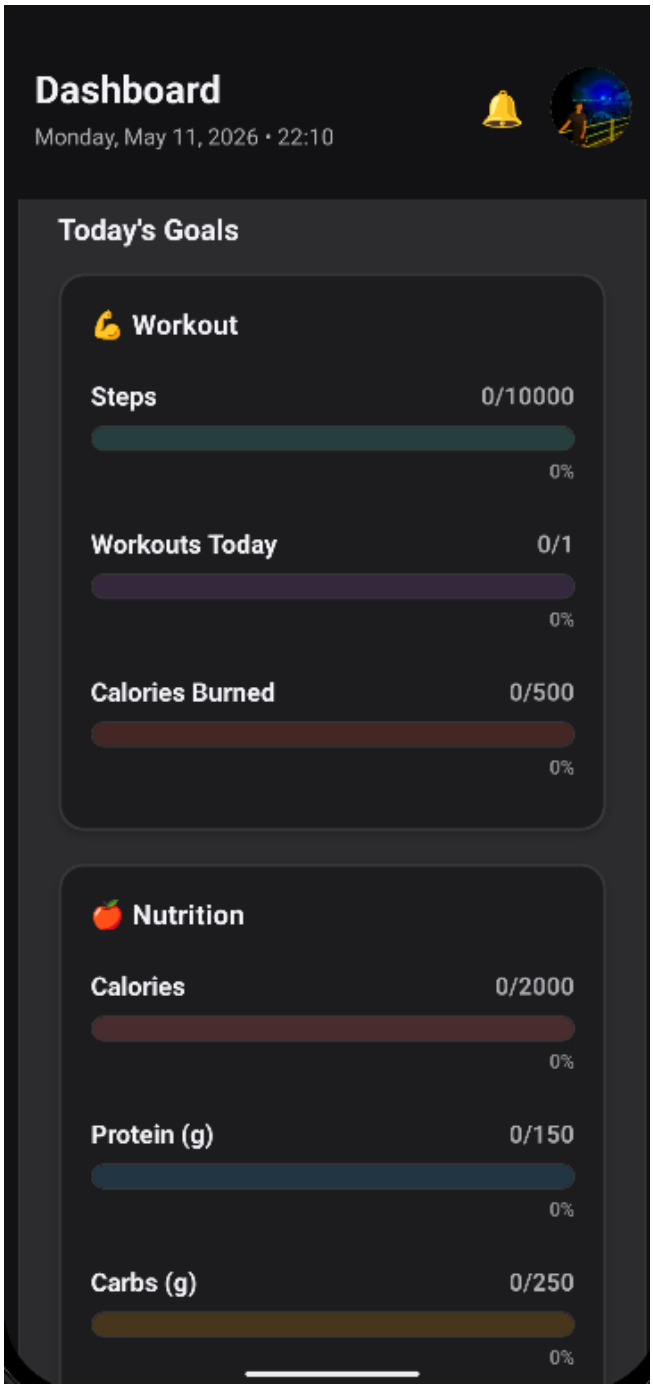

Figure 2.7: Dashboard overview displaying daily goals, nutrition targets, and progress indicators.

board is updated automatically through the nutrition and workout contexts, ensuring that recent activity is reflected throughout the application.

2.3.4 Profile Module

The profile module allows users to manage personal information such as age, weight, height, fitness goals, and daily activity targets. Changes made by the user are stored in Firestore, ensuring that profile information remains available across sessions.

The module also includes statistics related to workouts, calories burned, and nutrition progress. These values are updated using data provided by the *WorkoutContext* and *NutritionContext*, giving users an overview of their recent activity and progress.

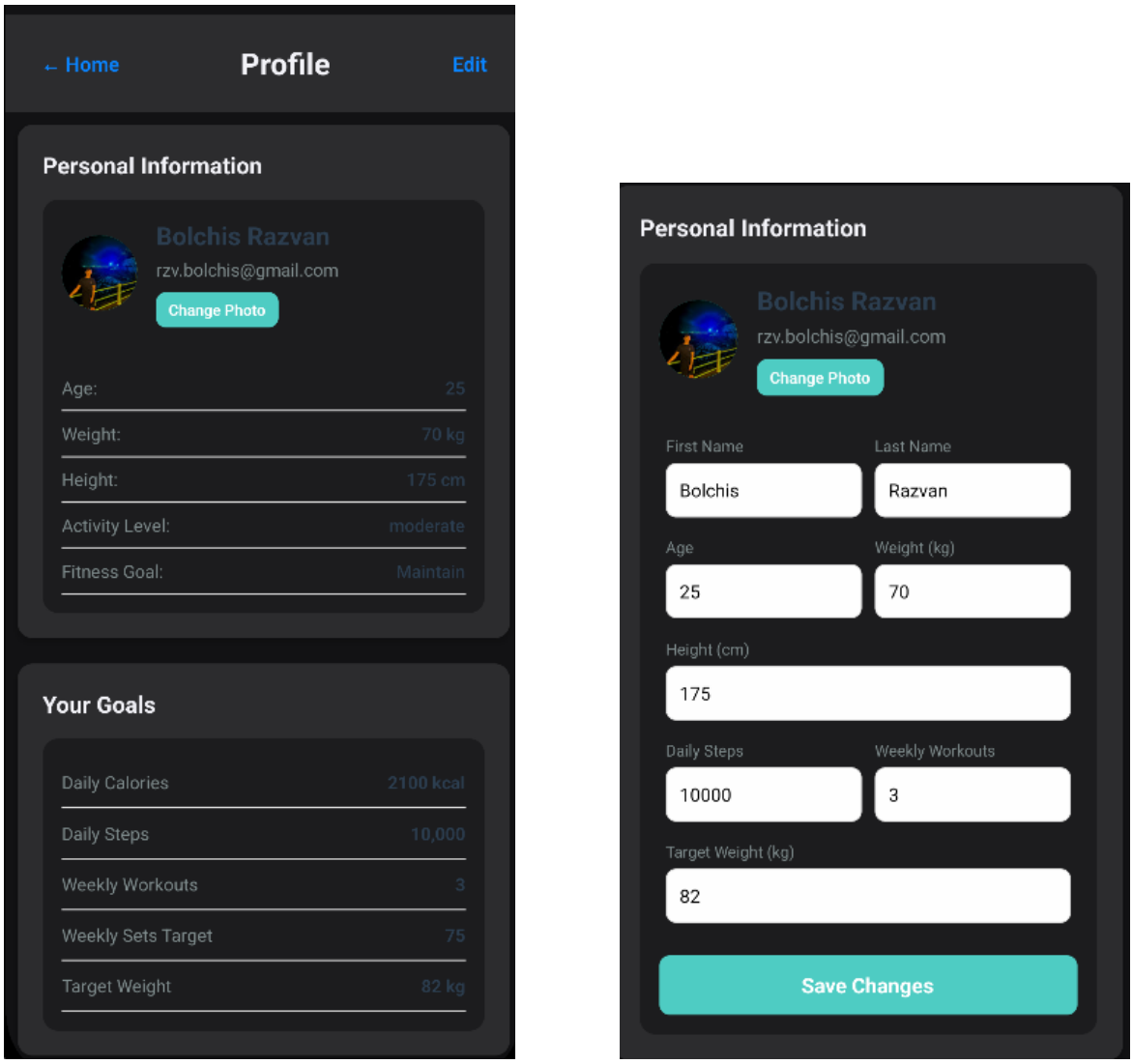

Figure 2.8: Profile overview and profile editing screens used to manage personal information and fitness goals.

Additional options include dietary preferences, privacy and GDPR information, and account management features. A reset-progress function is also available for users who want to clear their stored fitness and nutrition history.

2.3.5 Community Module

The community module adds social and motivational features to the application. Gamification features such as challenges, leaderboards, and badges are widely used to increase user engagement and motivation in fitness applications [HKS14, SASL17]. Users can create posts, share workout-related content, join challenges, view leaderboards, and unlock achievement badges. These features are designed to encourage regular participation and make progress more visible within the application.

The feed allows users to publish text, photos, or short videos, while challenge posts can also be shared directly from the challenges section. Challenges can include goals such as workout streaks, exercise counts, or calories burned. Leaderboards display weekly rankings based on workout activity, and badges provide visual rewards for completed achievements.

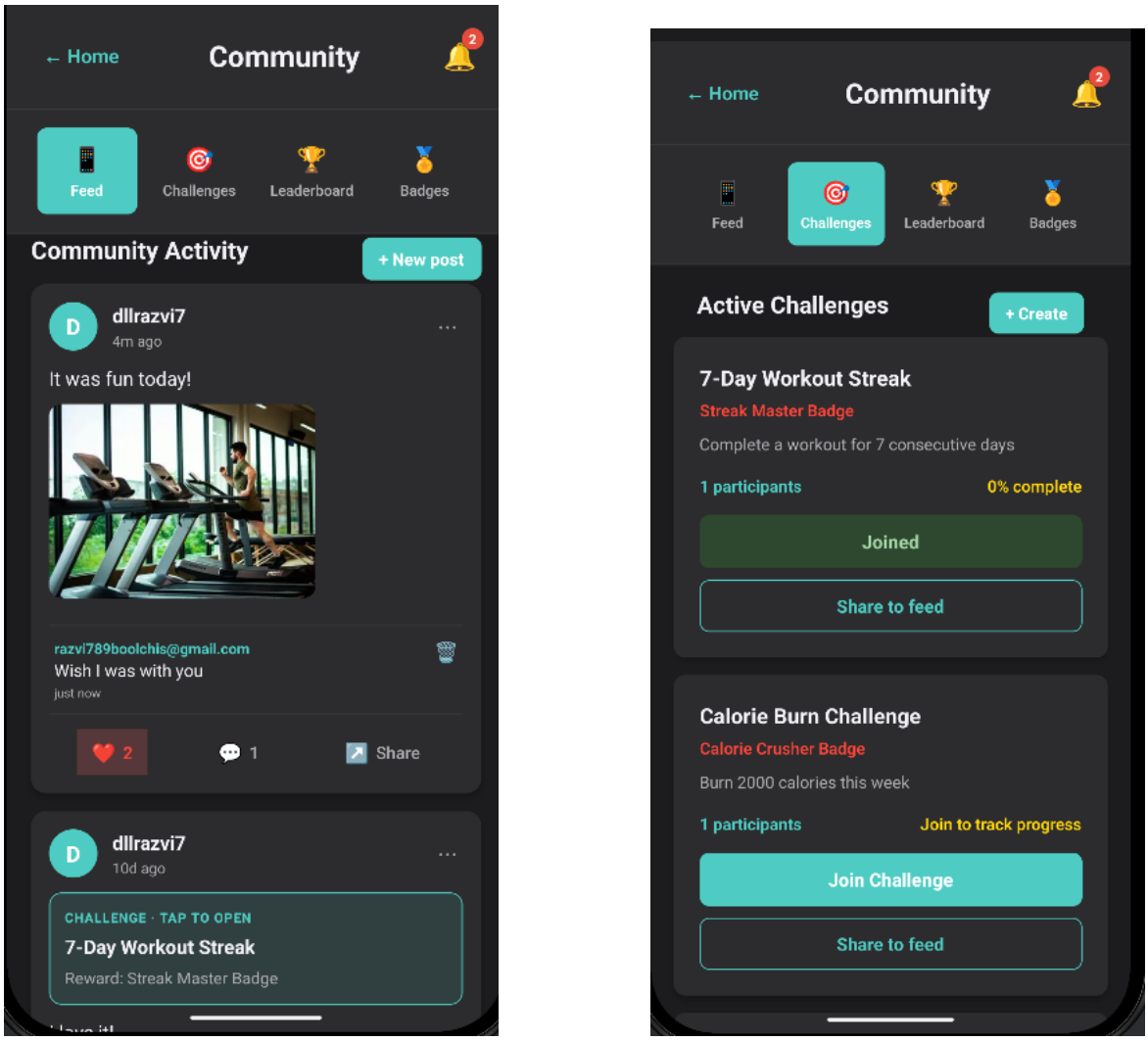

Figure 2.9: Community feed and active challenges used to support social interaction and user motivation.

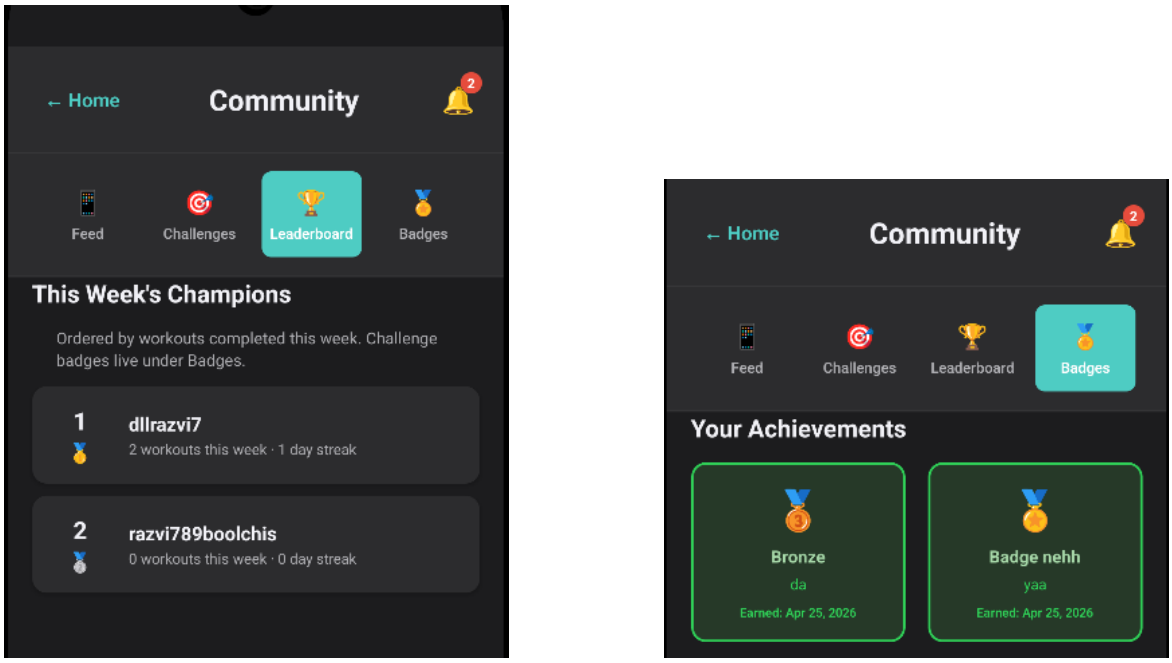

Figure 2.10: Leaderboard and achievement badges showing weekly rankings and completed challenge rewards.

2.4 Database Design and Data Models

Cloud Firestore is used as the main database for FitTrack. Data is organized around users and application modules, making it easier to store workout routines, nutrition logs, notifications, and community activity separately. This structure also allows the application to retrieve user-specific data efficiently and keep the main modules synchronized.

Path / Collection	Content	Purpose
`{uid}/routines`	Routine metadata, selected exercises, training parameters.	Personal workout routines.
`{uid}/workoutSessions`	Completed workouts, duration, sets, repetitions, calories.	Workout history and progress tracking.
`settings/weeklyPlanner`	Day-to-routine assignments.	Weekly workout planning.
`{uid}/notifications`	Notification messages and status.	In-app notification inbox.
`dailyNutritionLogs`	Meal entries, calories, macronutrients, timestamps.	Daily nutrition tracking.
`communityPosts`	Feed posts, likes, comments, shared challenges.	Social feed and interaction.
`communityChallenges`	Challenge definitions, participants, progress values.	Community challenges.
`publicLeaderboardStats`	Weekly workout statistics and ranking data.	Leaderboard display.

Table 2.3: Primary Firestore collections and their roles in FitTrack

Workout routine documents store the selected exercises together with information such as sets, repetitions, rest time, and exercise order. This allows saved routines to remain usable even when exercise data is later loaded from a different catalog source. Nutrition documents store meal entries with quantities, timestamps, and nutritional values, while community documents store posts, challenges, participation data, and leaderboard information.

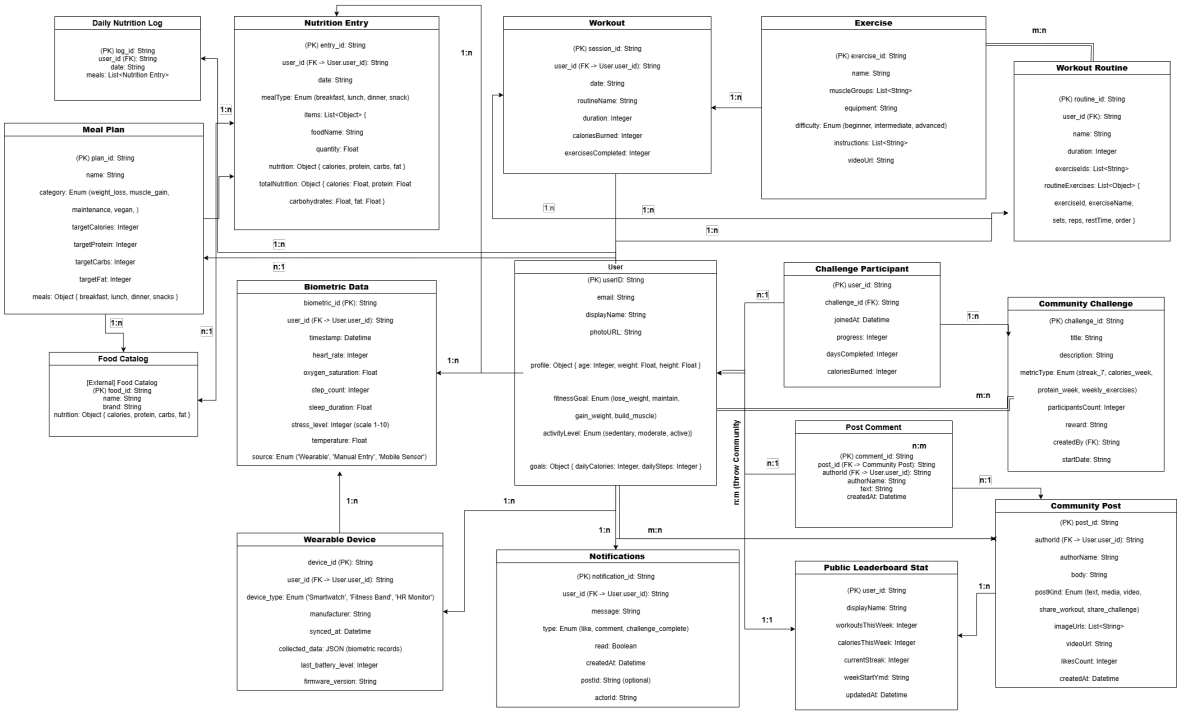

Figure 2.11: High-level database schema showing the main entities and relationships used by FitTrack.

2.5 Authentication and Security Implementation

FitTrack uses Firebase Authentication to manage user access and account security. Users can sign in using Google Sign-In (OAuth 2.0), a traditional email and password combination, or anonymous access for initial application exploration. Once authenticated, users can access personalized features such as workout routines, nutrition logs, challenges, and profile management.

Authentication status is monitored throughout the application, and protected screens are accessible only to logged-in users. This ensures that personal data and user-generated content remain associated with the correct account. Features that store information in Firestore, such as routine creation and workout tracking, require authentication before data can be saved.

Method	Technology	Purpose
Google Sign-In	OAuth 2.0	Fast login using an existing Google account.
Email / Password	Firebase Authentication	Traditional account registration and login.
Anonymous Access	Firebase Authentication	Allows users to explore the application before creating an account.

Table 2.4: Authentication methods supported by FitTrack

Security is provided through Firebase services and Firestore security rules. All communication between the application and cloud services is protected using encrypted connections, while Firestore rules restrict access to user-owned data. Sensitive configuration files, such as API credentials used for exercise catalog integration, are excluded from version control and stored locally during development.

FitTrack also follows several GDPR principles. Only information required for application functionality is collected, users are informed about the purpose of stored data, and account information can be exported when needed. Users may also request the deletion of their stored data, ensuring compliance with the right to erasure.

2.6 Testing and Validation

Testing was performed throughout the development process to verify that the main application features behaved as expected. Each module was tested individually, followed by integration testing to ensure that data was correctly shared across different parts of the application.

Authentication testing included account creation, login, logout, Google Sign-In integration, and validation of incorrect credentials. Nutrition features were tested by adding meals, calculating daily totals, applying filters, and verifying that data was correctly stored and retrieved from Firestore.

Workout testing focused on exercise discovery, muscle-group filtering, routine creation, session execution, timer functionality, and weekly workout planning. Additional testing ensured that workout results were correctly saved and reflected in the dashboard statistics.

The dashboard was tested to confirm that nutrition and workout information remained synchronized across the application. Profile management features were verified by updating personal information and fitness goals, while community testing

Module	Main Test Scenarios
Authentication	Login, registration, Google Sign-In, logout, invalid credentials
Nutrition	Food search, meal logging, calorie calculations, filtering and sorting
Workout	Exercise filtering, routine creation, workout execution, timers, weekly planner
Dashboard	Progress calculations, synchronization with nutrition and workout data
Profile	Updating personal information, fitness goals, preferences
Community	Creating posts, joining challenges, notifications, leaderboard updates

Table 2.5: Main testing scenarios performed for FitTrack modules

included post creation, challenge participation, notifications, likes, and leaderboard updates.

Navigation between screens, Context API state management, and Firestore persistence were also tested to ensure that data remained consistent during normal application usage.

2.7 Implementation Status and Milestones

At the time of writing, the main features planned for FitTrack have been implemented and integrated into a functional mobile application. The project includes nutrition tracking, workout management, personalized meal plans, community features, user profiles, authentication, and cloud-based data persistence through Firebase.

Feature Area	Status
Authentication and User Accounts	Completed
Nutrition Tracking	Completed
Meal Plans	Completed
Workout Management	Completed
Weekly Planner	Completed
Dashboard and Statistics	Completed
Profile Management	Completed
Community Features	Completed
Notifications	Completed
Firebase Integration	Completed

Table 2.6: Implementation status of the main FitTrack modules

The nutrition module supports food discovery through search, filtering, and sort-

ing options, together with meal logging, calorie tracking, macronutrient monitoring, and progress visualization. Users can also follow predefined meal plans and export nutrition data for later analysis.

The workout module includes exercise discovery through external exercise catalogs, muscle-group filtering using an interactive body map, personal workout routines, guided workout sessions with timers, weekly planning, and workout history tracking. Exercises can be added directly to existing routines or used to create new routines from the exercise details screen.

The dashboard provides a centralized overview of fitness and nutrition progress, while the profile module allows users to manage personal information, fitness goals, and application preferences. Both modules are synchronized with the application's shared contexts to ensure that information remains consistent across screens.

Community functionality includes challenges, leaderboards, badges, social posts, and notifications. These features encourage user engagement and allow users to share progress and participate in friendly competition.

Throughout development, several interface improvements were made to simplify navigation, improve consistency between screens, and enhance the overall user experience. Data synchronization between modules is handled through Context API and Firebase services, allowing updates to be reflected across the application in real time.

2.8 Limitations and Future Work

Although the current version of FitTrack provides the main functionality required for fitness tracking, nutrition management, and community interaction, several improvements could further enhance the application.

Current limitations. Exercise data depends on external catalog providers, which means that internet connectivity is required for accessing the complete exercise library. In addition, workout routine management could be expanded with more advanced editing features, while offline support is currently limited to the capabilities provided by Firebase.

Area	Potential Improvements
Workout Management	Full catalog search, pagination, advanced routine editing, and exercise reordering.
Analytics	Weekly summaries, training trends, muscle-group distribution, and long-term progress tracking.
Wearables and Sensors	Heart rate integration, sleep tracking, recovery indicators, and activity synchronization.
Community Features	Friend groups, richer challenges, workout sharing, and improved achievement systems.
Nutrition	Adaptive meal plans, personalized food recommendations, and grocery or meal-order integrations.
Artificial Intelligence	Workout recommendations, nutrition suggestions, and behavior-based personalization.
Computer Vision	Exercise form analysis and real-time movement feedback using the device camera.
Offline Support	Local storage, deferred synchronization, and conflict resolution mechanisms.

Table 2.7: Potential future extensions of the FitTrack platform

Several future directions are particularly promising. Integration with wearable devices could provide additional health data such as heart rate, activity levels, and sleep duration, allowing FitTrack to generate more personalized recommendations. Advanced analytics could help users better understand long-term trends in both training and nutrition.

The community system could also be expanded through friend groups, richer challenge formats, and social sharing features. In addition, nutrition planning could move beyond simple tracking by introducing adaptive meal plans and personalized food recommendations.

Looking further ahead, technologies such as computer vision and lightweight machine learning could be explored. These could enable exercise-form feedback through the device camera and more personalized workout or nutrition suggestions based on user behavior and progress history.

The modular architecture of FitTrack was designed to support these extensions, allowing future features to be integrated without major changes to the existing application structure.

2.9 Chapter Summary

This chapter presented the technical implementation of FitTrack, including the technologies used, the overall application architecture, and the main modules developed during the project. The roles of React Native, TypeScript, Firebase, and Firestore were discussed together with the architectural decisions that support data synchronization and communication between modules.

The nutrition, workout, dashboard, profile, and community modules were described in terms of their functionality and interaction with shared application data. Particular attention was given to exercise catalog integration, workout routine management, nutrition tracking, and the use of Firebase services for authentication and cloud data storage.

The chapter also presented the database structure, authentication mechanisms, testing process, implementation status, and the main limitations of the current version of the application. Finally, several future development directions were identified, including wearable integration, advanced analytics, improved offline support, and additional personalization features.

Together, these components form the technical foundation of FitTrack and provide a scalable basis for future development and feature expansion.

Chapter 3

User Interface Design and Implementation

The user interface plays an important role in the overall experience of a fitness application. Users interact with the interface every time they log meals, start workouts, check their progress, or participate in community activities. Because of this, the interface must present information clearly, reduce unnecessary complexity, and allow common actions to be completed quickly.

In FitTrack, particular attention was given to keeping navigation intuitive and maintaining a consistent visual style across all modules. The interface was designed to provide immediate access to frequently used features while still displaying progress information, statistics, and notifications in an organized manner. Since many interactions take place during workouts or while users are on the move, screens were designed to minimize the number of steps required to complete common tasks.

The visual design follows Material Design principles and adapts them to the needs of a fitness and nutrition application. Particular attention was given to usability, consistency, and accessibility, which are commonly recognized as important quality attributes in mobile health applications [SHKW15, SGL+21]. React Native was used to implement a consistent interface across supported mobile platforms, while the theme system allows users to switch between light and dark modes according to their preferences.

3.1 Design Philosophy and Principles

The design of FitTrack focuses on ease of use and clarity. Since users often interact with the application while exercising, tracking meals, or quickly checking their progress, the interface was designed to minimize unnecessary complexity and make

important actions easy to access.

Throughout development, attention was given to keeping screens visually consistent, providing clear feedback after user actions, and ensuring that information remains easy to read regardless of the amount of data displayed. The goal was to create an interface that feels familiar across all modules while helping users stay focused on their fitness objectives.

3.1.1 Core Design Principles

Several principles guided the design of the interface and influenced decisions related to layout, navigation, visual styling, and interaction patterns. These principles helped maintain a consistent user experience across the dashboard, nutrition, workout, profile, and community modules.

Principles applied in FitTrack:

- *Simplicity* - important information is displayed clearly, while unnecessary elements are avoided to reduce visual clutter.
- *Consistency* - similar layouts, colors, icons, and navigation patterns are used throughout the application to make screens easier to understand.
- *Feedback* - actions such as saving data, completing workouts, joining challenges, or updating settings provide immediate visual confirmation.
- *Accessibility* - sufficient contrast, appropriate spacing, and large touch targets help improve usability on mobile devices.

3.1.2 Visual Hierarchy and Information Architecture

The overall structure of FitTrack is centered around the dashboard, which acts as the main entry point to the application's features. From the dashboard, users can quickly access the nutrition, workout, profile, and community modules while also viewing a summary of their daily progress.

The navigation system was designed to remain consistent across the application. Similar layouts, icons, and interaction patterns are used throughout different screens, helping users move between modules without having to learn new navigation flows for each section.

Visual hierarchy is created through the use of typography, spacing, color, and component size. Important information such as progress indicators, daily goals, and primary actions is placed in prominent positions, while secondary actions receive less visual emphasis. Progress is displayed using circular indicators and progress

bars, allowing users to quickly understand their current status without reading large amounts of text.

3.2 Screen-Level Design and Implementation

FitTrack consists of several main screens, each designed around a specific area of functionality. Although the content differs between modules, all screens follow the same visual style and interaction patterns. This consistency helps users navigate the application more easily while reducing the learning curve for new features.

3.2.1 Dashboard Screen

The dashboard serves as the main screen of the application and provides a quick overview of the user's daily fitness and nutrition progress. Centralized dashboards are widely used in health and fitness applications because they support self-monitoring and help users maintain awareness of their progress over time [SHKW15]. Its purpose is to bring together the most important information in a single place, allowing users to check their current status without navigating through multiple screens.

Four circular progress indicators display key metrics related to daily activity, including calories consumed, calories burned, step count, and workout activity. Additional progress bars provide a more detailed view of nutrition targets and activity goals.

Main dashboard components:

- *Activity indicators* - visual summaries of daily fitness and nutrition progress.
- *Nutrition progress bars* - detailed information about calories and macronutrient targets.
- *Quick actions* - shortcuts for frequently used features such as food logging, workout tracking, profile access, and community interaction.

The dashboard is synchronized with both the *NutritionContext* and *WorkoutContext*, ensuring that newly logged meals, completed workouts, and updated statistics are reflected immediately throughout the application.

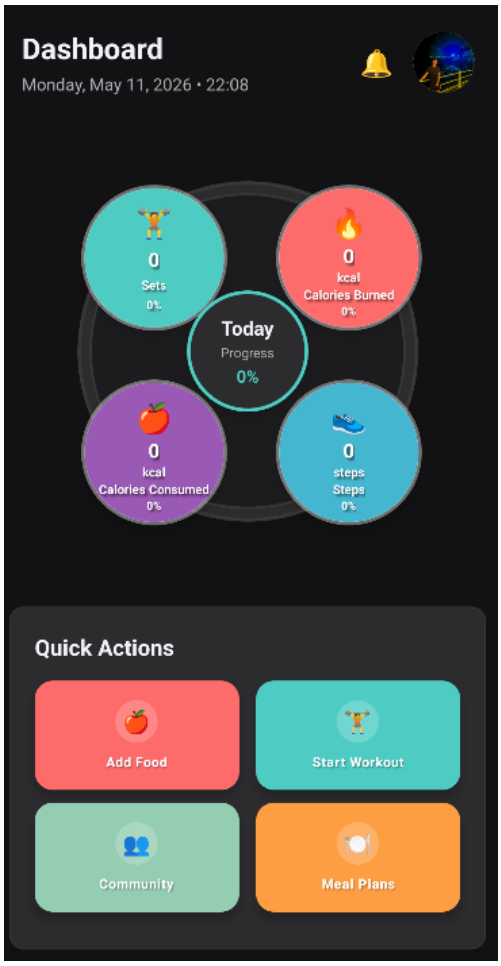

Figure 3.1: Dashboard screen displaying daily activity indicators, nutrition progress, and quick-access actions.

3.2.2 Nutrition Screen

The nutrition screen allows users to search for foods, log meals, and monitor their daily nutritional intake. The interface combines food discovery, meal tracking, and progress monitoring within a single workflow, making it easy to record meals and compare them against daily goals.

Users can search for foods using a dedicated search bar and refine results through sorting and filtering options. Food items display key nutritional information, including calories, protein, carbohydrates, and fat. Selecting an item opens a detailed screen where quantities can be adjusted and nutritional values are recalculated automatically.

Meal entries are organized into categories such as breakfast, lunch, dinner, and snacks. Daily totals are updated in real time and displayed together with progress indicators, helping users monitor their calorie and macronutrient targets throughout the day.

Component	Functionality	Implementation Details
Search Bar	Real-time filtering	Debounced input with server-side query
Sort Controls	Calorie/macro ordering	Toggles with ascending/descending modes
Filter Toggles	Dietary restrictions	Boolean filters for vegan/vegetarian
Food Cards	List item rendering	FlatList with virtualization
Detail View	Full nutrition + quantity	Modal overlay with live recalculation
Daily Journal	Meal-time segmentation	Sectioned list with swipe-to-delete
Progress Footer	Goal adherence	Persistent bar with running totals

Table 3.1: Nutrition screen components and implementation approaches

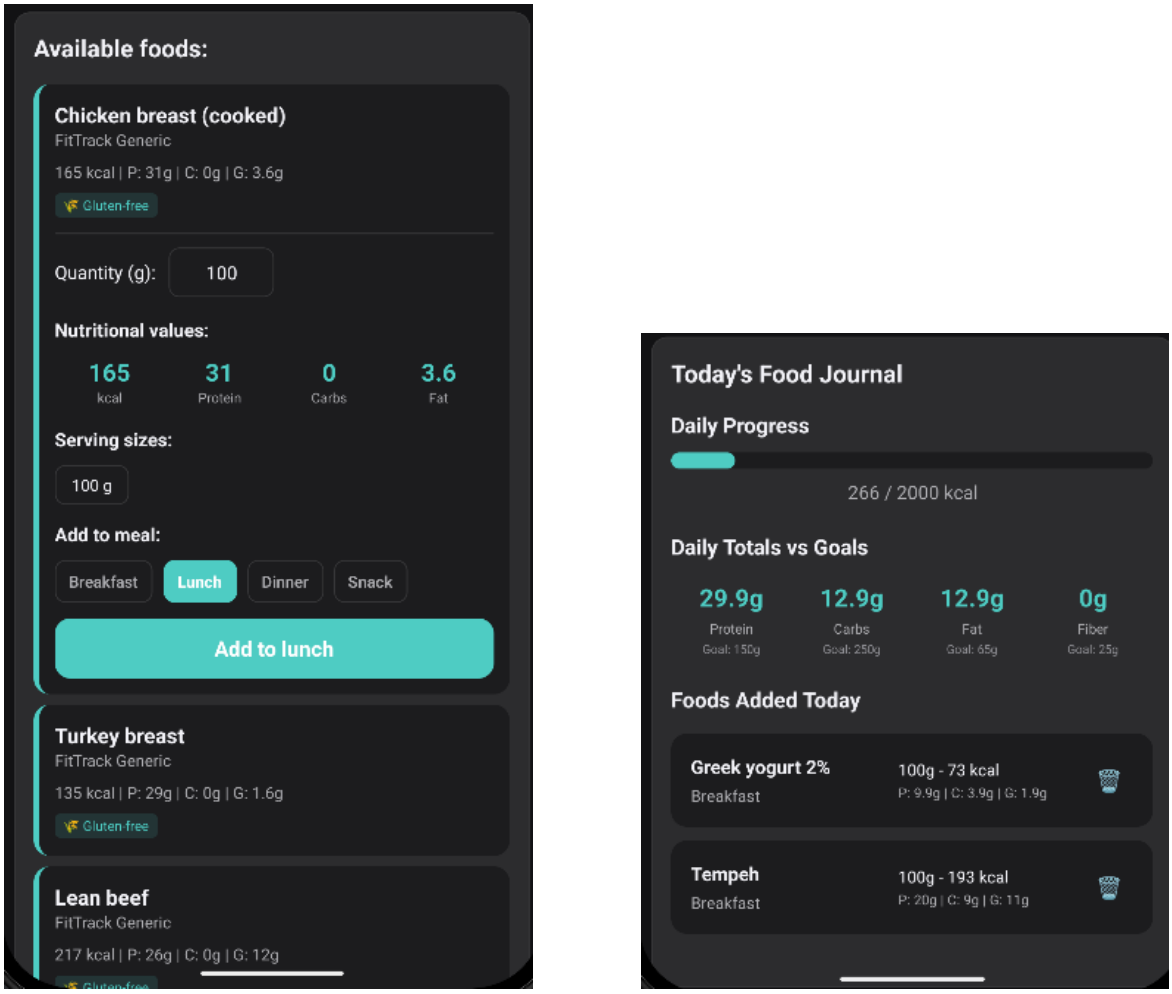

Figure 3.2: Food selection interface and daily nutrition journal with calorie and macronutrient tracking.

3.2.3 Workout Screen

The workout screen combines exercise discovery, routine management, and guided workout execution within a single module. Users can browse a large exercise catalog and refine results through muscle-group selection, difficulty levels, and equipment filters. An interactive body map provides a visual method for selecting target muscles, making exercise discovery more intuitive than traditional text-based filtering.

Exercise cards present essential information such as exercise name, target muscle group, equipment, and difficulty level. Selecting an exercise opens a dedicated detail screen containing instructions, demonstration media, and options for adding the exercise to a personal workout routine.

Component	**Purpose**
Interactive Body Map	Allows users to filter exercises visually by muscle group.
Exercise Catalog	Displays available exercises with filtering options.
Exercise Details	Provides instructions, media demonstrations, and exercise information.
Personal Routines	Enables creation and management of custom workout plans.
Workout Session	Guides users through exercises, sets, and rest periods.
Weekly Planner	Supports scheduling routines across the week.

Table 3.2: Main components of the workout screen

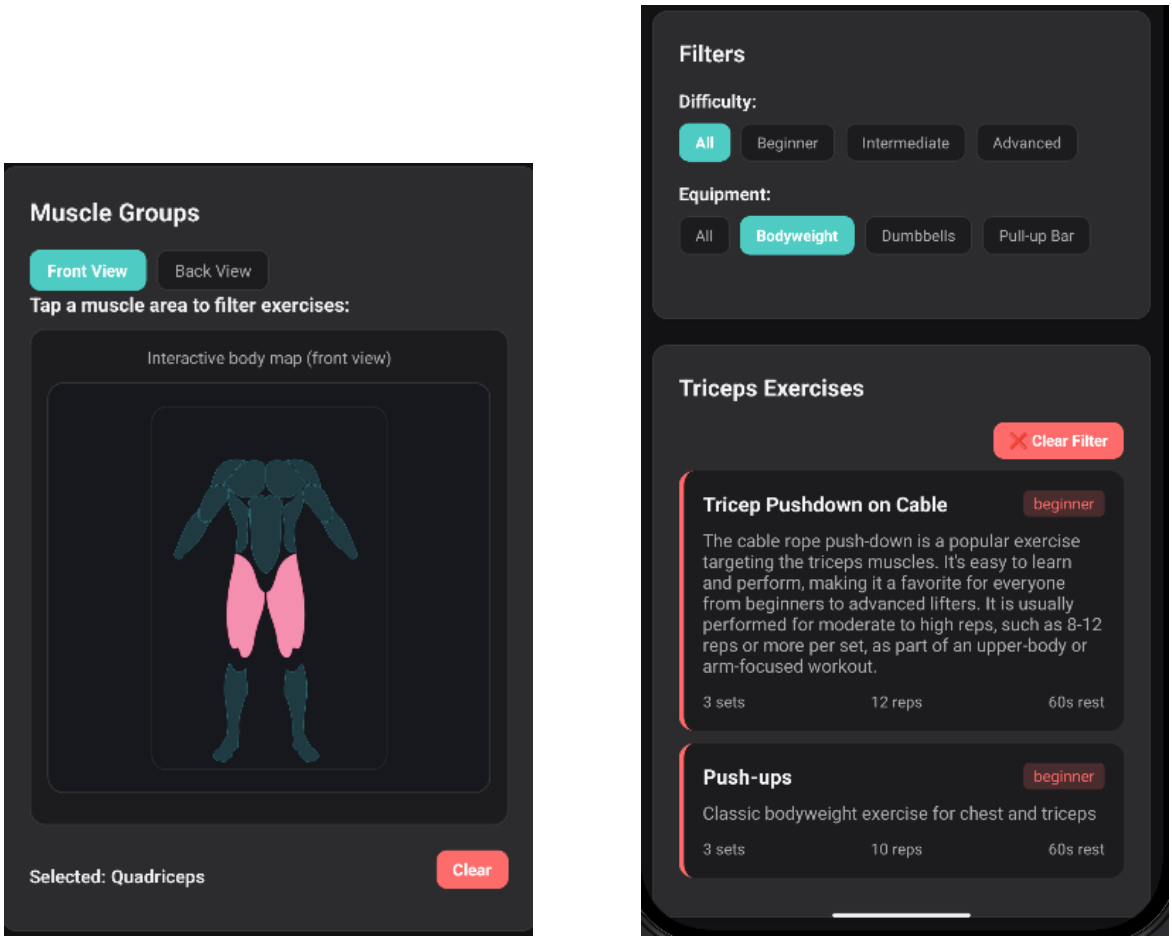

Figure 3.3: Interactive muscle-group selection and exercise catalog browsing.

The module also supports personal routine creation and guided workout execution. During active sessions, users receive visual feedback for completed sets and rest periods through integrated countdown timers. Workout results are stored automatically and contribute to progress tracking throughout the application.

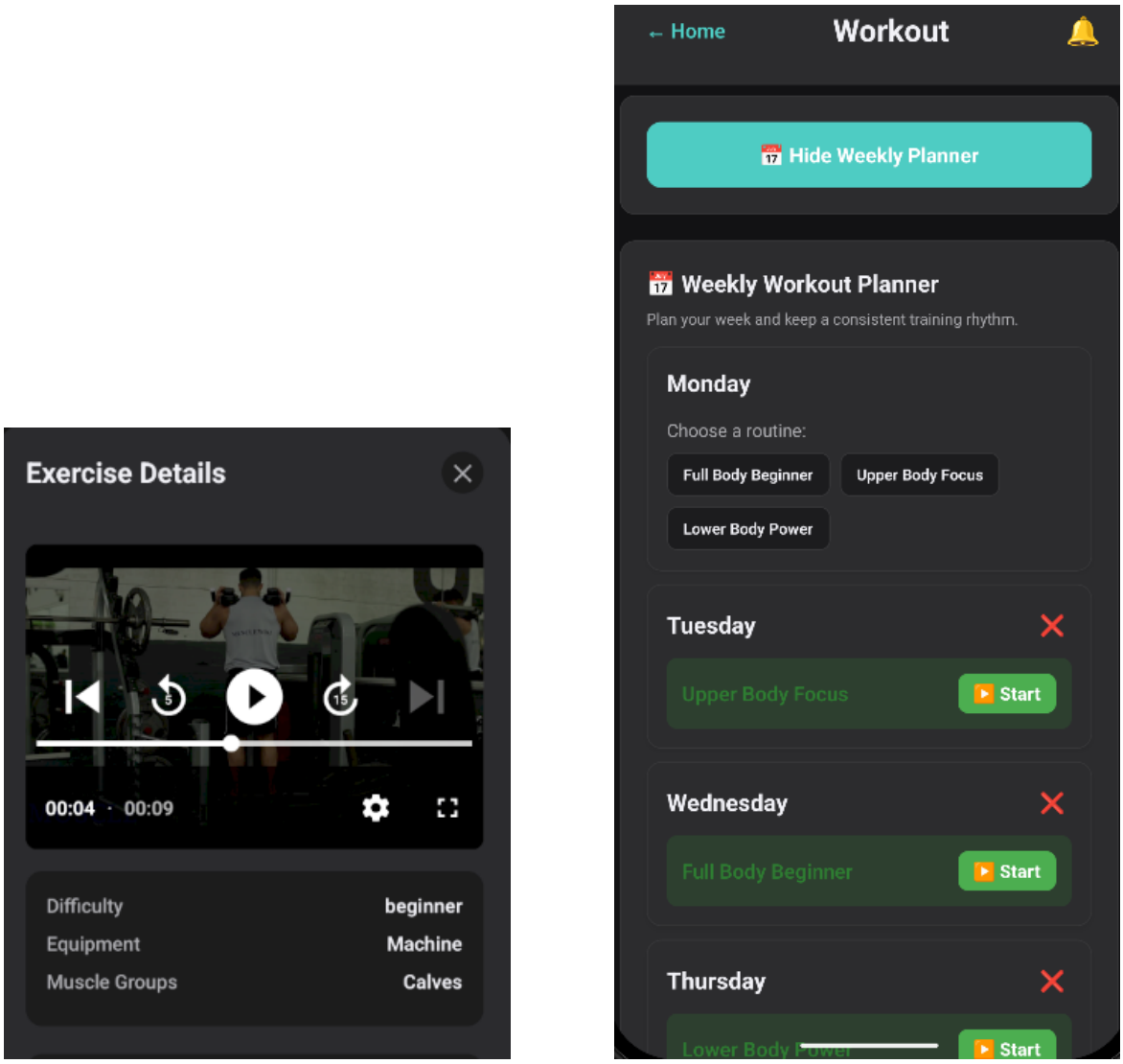

Figure 3.4: Exercise details view and weekly workout planning interface.

3.2.4 Profile Screen

The profile screen centralizes personal information, fitness goals, and application preferences. Users can view and edit attributes such as age, weight, height, activity level, and target goals through a dedicated editing interface. Changes are synchronized with Firestore, ensuring that profile data remains available across sessions and devices.

The screen also provides access to workout and nutrition statistics, including completed workouts, calories burned, activity streaks, and personalized targets. Additional settings allow users to manage preferences such as dietary restrictions, theme selection, and privacy-related options.

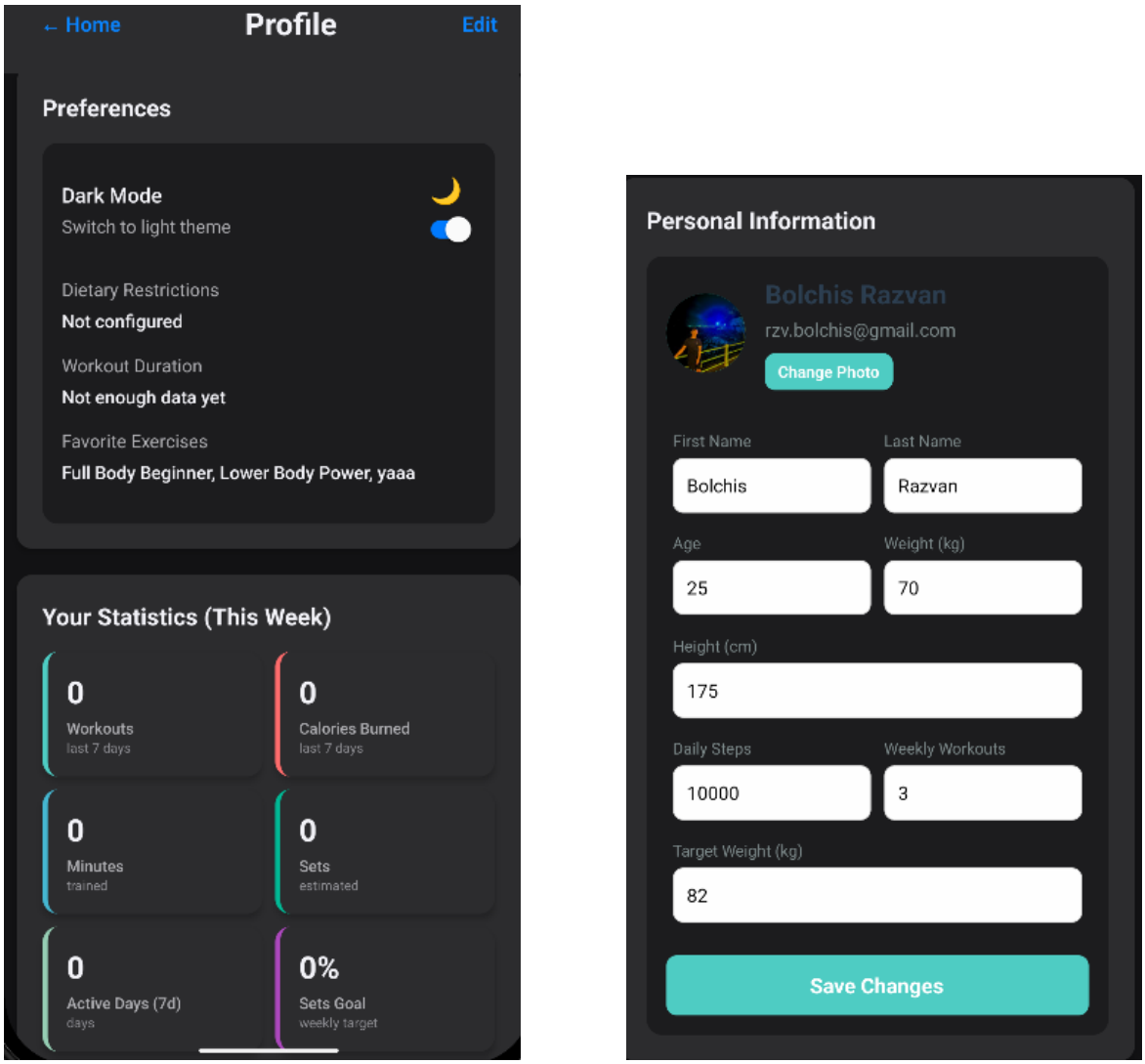

Figure 3.5: Profile overview and profile editing interface used to manage personal information and fitness goals.

3.2.5 Community Screen

The community screen introduces social and motivational features designed to increase user engagement. Functionality is organized into four dedicated sections: Feed, Challenges, Leaderboard, and Badges. Social features such as challenges, leaderboards, and achievement systems are frequently used in fitness applications to improve motivation and long-term engagement [HKS14, SASL17].

The feed allows users to share workout-related content and interact through likes and comments. Challenges encourage participation through goal-based activities that can be joined directly from the application. Progress is tracked automatically and reflected in leaderboards and achievement badges.

Real-time updates ensure that new posts, challenge participation, and notifications are reflected immediately throughout the community module.

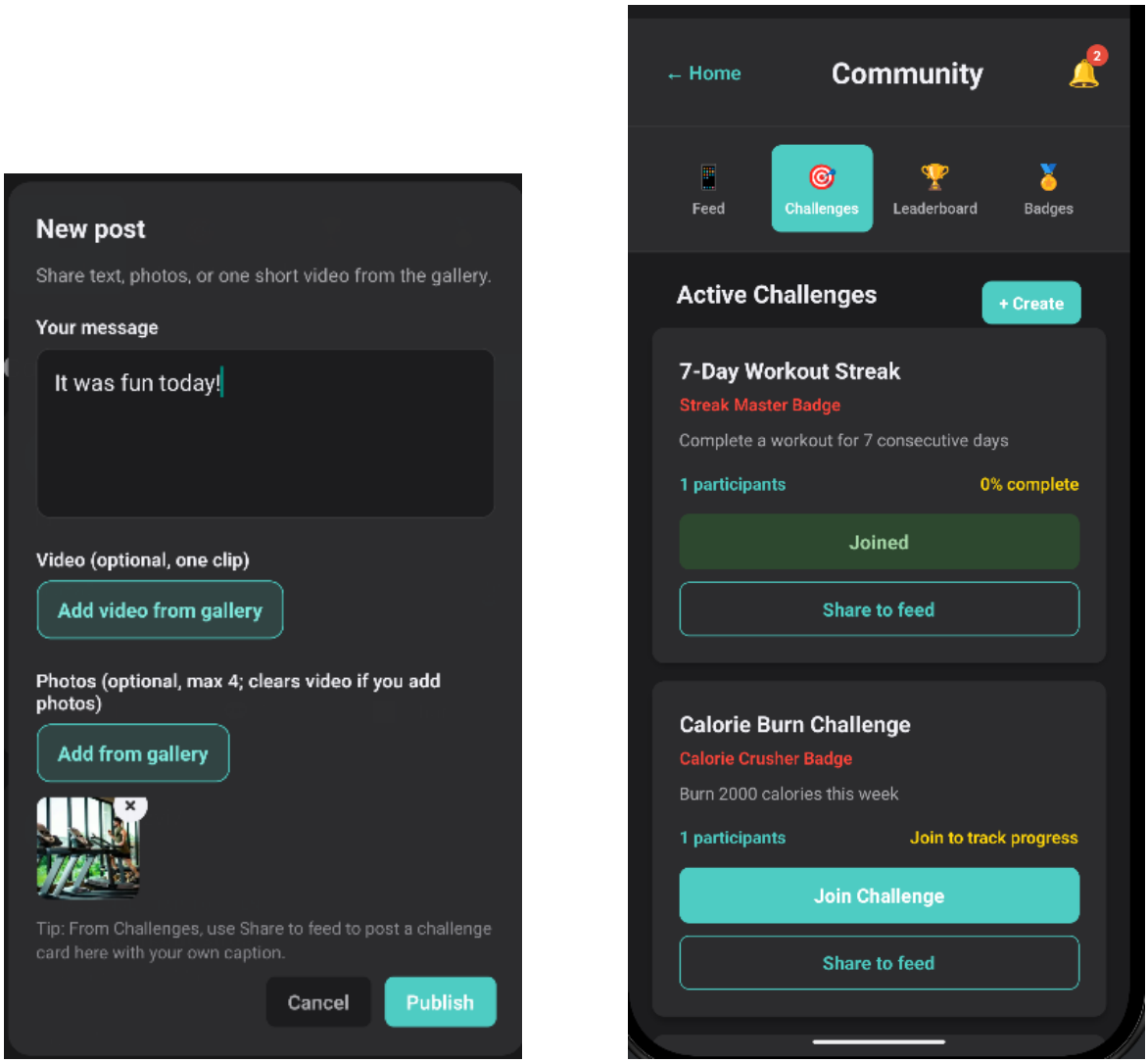

Figure 3.6: Community feed and challenge management interfaces supporting social interaction and participation.

Tab	Primary Functionality	Key Features
Feed	Social activity stream	Infinite scroll, like/comment
Challenges	Fitness competitions	Join/leave, progress tracking, leaderboard preview
Leaderboard	Competitive rankings	Real-time updates, top-3 emphasis
Badges	Achievement visualization	Unlock animations, progress indicators

Table 3.3: Community screen tabs and core features

3.3 Navigation Implementation

Navigation in FitTrack is implemented using React Navigation and combines stack, tab, and modal navigators. This structure allows users to move efficiently between modules while preserving state and maintaining predictable navigation behavior across the application.

3.3.1 Navigation Patterns and Hierarchies

The navigation hierarchy follows an authentication-first approach. Users must successfully authenticate before accessing the main application. After login, the interface transitions to a tab-based navigation structure that provides direct access to the primary modules: Dashboard, Nutrition, Workout, Profile, and Community.

Each module contains its own internal stack navigator, allowing users to move between overview screens, detail views, and configuration screens without affecting navigation in other sections. Modal screens are used for short-lived interactions such as quantity adjustments, challenge confirmations, and workout-related actions.

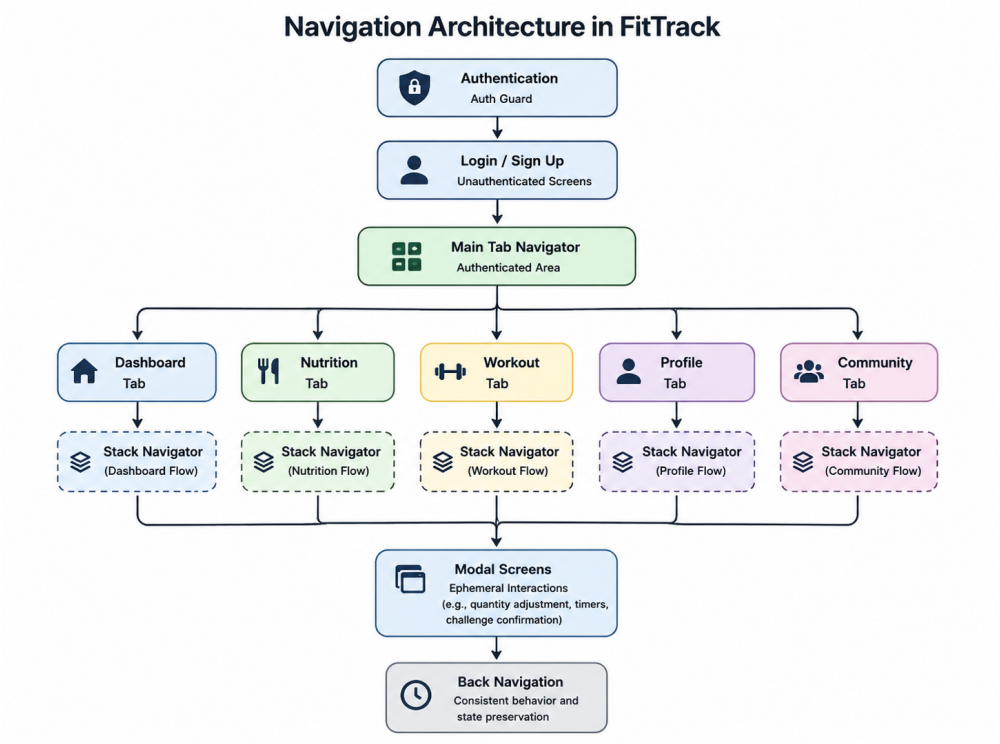

Figure 3.7: Navigation hierarchy showing authentication flow, tab navigation, stack navigators, and modal interactions.

Navigation constructs used:

- *Stack Navigator* for hierarchical screen flows and detail views.

- *Tab Navigator* for rapid switching between primary application modules.
- *Modal Screens* for temporary interactions that should not interrupt the main workflow.
- *Authentication Guards* for restricting access to protected content based on session state.

3.3.2 State Preservation and Deep Linking

FitTrack preserves important navigation information between sessions, such as the active tab and the current screen within a module. This allows users to return to the application without losing their general position in the navigation flow.

Temporary interface states, such as open modals or incomplete form inputs, are not preserved in order to avoid displaying outdated information when the application is reopened. Deep links are used for actions such as opening a notification directly in the related screen, for example a specific community challenge. Protected screens still require authentication before they can be accessed.

3.4 Component Design, Layout, and Responsiveness

The interface is built using reusable components that share common styling and behavior across the application. This approach reduces repeated code and helps maintain a consistent visual experience between modules.

3.4.1 Core Reusable Components

Shared component set:

- `Card` - used as a consistent container for foods, exercises, challenges, and profile sections.
- `ProgressBar` - used for nutrition goals, workout progress, and challenge completion.
- `CircularProgress` - used for the dashboard indicators showing daily progress.
- `Button` - provides consistent sizing, styling, and states such as enabled, disabled, and loading.
- `TextInput`, `Select`, and `Toggle` - used for forms, preferences, and editable user settings.

These shared components make the interface easier to maintain and help ensure that similar actions look and behave the same across the application. They are used in several areas, including profile editing, routine creation, nutrition logging, and community interactions.

3.4.2 Layout Patterns and Responsive Design

FitTrack uses responsive layouts to provide a consistent experience across different screen sizes. The interface is built with Flexbox, allowing content to adapt automatically to available space.

On smartphones, information is displayed through sequential navigation, where users move between screens as needed. On larger devices such as tablets, layouts can present lists and detailed views side by side, reducing navigation steps and making better use of screen space.

Typography respects system accessibility settings, allowing text to scale according to user preferences. Flexible containers and automatic text wrapping help preserve readability and prevent interface elements from overlapping on smaller screens.

3.5 Theme System and Dark Mode

FitTrack supports both light and dark themes through a centralized *ThemeContext*. Users can manually select their preferred appearance from the Profile screen or allow the application to follow the operating system theme automatically. Supporting user preferences and adapting the interface to different usage contexts are considered important aspects of user-centered digital health design [NLZ+24].

3.5.1 Theme Architecture

The theming system is based on reusable design tokens, including colors, spacing values, typography settings, and shadow definitions. Components reference these tokens rather than hard-coded values, allowing the entire interface to update immediately when the theme changes.

Special attention was given to dark mode design. Instead of using pure black and white, softer shades were selected to reduce eye strain while maintaining sufficient contrast and readability.

3.5.2 Theme Switching and Persistence

Theme preferences are stored locally and restored automatically when the application is reopened. When a theme change occurs, the interface updates through the

Context API and applies a short transition animation to provide a smoother visual experience.

Theme Element	Light Mode	Dark Mode
Background	White (#FFFFFF)	Dark gray (#121212)
Surface	Light gray (#F5F5F5)	Elevated dark (#1E1E1E)
Primary	Blue (#2196F3)	Light blue (#64B5F6)
Text Primary	Very dark gray (#212121)	Very light gray (#E0E0E0)
Text Secondary	Medium gray (#757575)	Medium gray (#9E9E9E)

Table 3.4: Theme color tokens for Light and Dark modes

3.6 Visual Feedback and Animation

Visual feedback plays an important role in helping users understand the results of their actions and maintain awareness of application state. FitTrack uses animations, transitions, and temporary notifications to provide feedback without interrupting the user experience.

3.6.1 Interaction Feedback

Interactive elements respond immediately to user input. Buttons briefly change size when pressed, helping users confirm that an action has been registered. List items highlight before opening detailed views, while swipe gestures reveal contextual actions through smooth animations.

Forms display loading indicators during longer operations such as authentication or data synchronization. After completion, success and error messages provide clear feedback about the outcome of the action.

3.6.2 State Transition Animations

Navigation transitions follow platform conventions to create a familiar user experience. Stack navigation typically uses slide animations, while tabs and modal windows rely on lighter transition effects.

Progress indicators animate smoothly when values change, making updates easier to follow. Temporary notifications (toasts) appear for actions such as adding a meal, joining a challenge, or completing a workout, then disappear automatically after a short delay.

3.7 Accessibility and Inclusive Design

Accessibility considerations were incorporated throughout the design process to ensure that FitTrack remains usable for a wide range of users. Human-centered design research highlights accessibility and usability as essential factors influencing adoption and continued use of digital health applications [SGL+21, NLZ+24].

3.7.1 Screen Reader Support

Interactive controls include descriptive accessibility labels, allowing screen readers to communicate their purpose clearly. Informative images are accompanied by alternative descriptions, while decorative visual elements are ignored by assistive technologies.

Navigation controls use explicit labels so users can easily understand where an action will lead.

3.7.2 Motor Accessibility

Touch targets follow platform guidelines, ensuring that buttons and interactive elements remain easy to activate. Adequate spacing between controls reduces accidental taps, particularly on smaller screens.

Where appropriate, alternative interactions such as long-press actions are provided in addition to gesture-based controls, improving accessibility for users with limited motor precision.

3.7.3 Cognitive Accessibility

The interface minimizes cognitive load through progressive disclosure, presenting detailed information only when it becomes relevant. Clear language is preferred over technical terminology, while contextual explanations help users understand less familiar concepts.

Error messages are designed to be informative and actionable, guiding users toward a solution rather than simply reporting that a problem occurred.

Accessibility measures implemented in FitTrack:

- Color contrast validated across both light and dark themes.
- Typography that scales according to system accessibility settings.
- Consistent navigation patterns throughout the application.
- Clearly labeled interactive elements for assistive technologies.

- Adequate spacing and touch target sizes to reduce input errors.

3.8 Chapter Summary

This chapter presented the design and implementation of FitTrack's user interface, including the main design principles, navigation structure, reusable components, theming system, and accessibility considerations. The dashboard, nutrition, workout, profile, and community screens were discussed from both a visual and functional perspective, highlighting the design decisions that support everyday user interactions.

Particular attention was given to maintaining consistency across screens, simplifying navigation, and providing clear feedback through visual indicators, animations, and notifications. The responsive layout and theme system help ensure a comfortable experience across different devices and user preferences, while accessibility features improve usability for a broader range of users.

Together with the technical architecture presented in Chapter 2, the interface design forms a cohesive environment for fitness tracking, nutrition management, and community engagement. The result is a mobile application that aims to make daily health-related activities easier to monitor, organize, and maintain over time.

Chapter 4

Implementation Results and Evaluation

This chapter presents the results obtained during the development of FitTrack, including the implemented features, testing activities, performance observations, and the main challenges encountered throughout the project. The objective is to evaluate how the application performs in practice and how successfully the goals defined in Chapter 1 were achieved.

The development process followed an iterative approach. Core functionality related to nutrition tracking, workout management, dashboard visualization, profile management, and community interaction was implemented first and then refined through testing and continuous improvements. This approach made it possible to validate interactions between modules early in the development process while gradually expanding the application's functionality and usability.

The following sections examine the implemented features, testing procedures, performance considerations, current limitations, and opportunities for future improvement.

4.1 Implementation Status and Feature Coverage

The current version of FitTrack includes the main functionality planned at the beginning of the project. Core features related to nutrition tracking, workout management, progress monitoring, profile customization, and community interaction have been successfully implemented and integrated into a single application.

The nutrition module supports food search, meal logging, calorie tracking, macronutrient monitoring, and meal-plan browsing. The workout module provides exercise discovery, personal routine creation, guided workout sessions, and weekly workout planning. Progress from both modules is reflected in the dashboard, which serves

as the central overview screen of the application. Self-monitoring and progress visualization are frequently associated with improved engagement and adherence in digital health applications [SHKW15].

Profile management allows users to customize personal information, goals, and preferences, while community features encourage engagement through challenges, leaderboards, badges, and social interactions. Previous research has shown that gamification mechanisms and social competition can positively influence user motivation and participation in health-related applications [HKS14, SASL17]. Data synchronization is handled through Firebase services, ensuring that user information and activity remain available across application sessions.

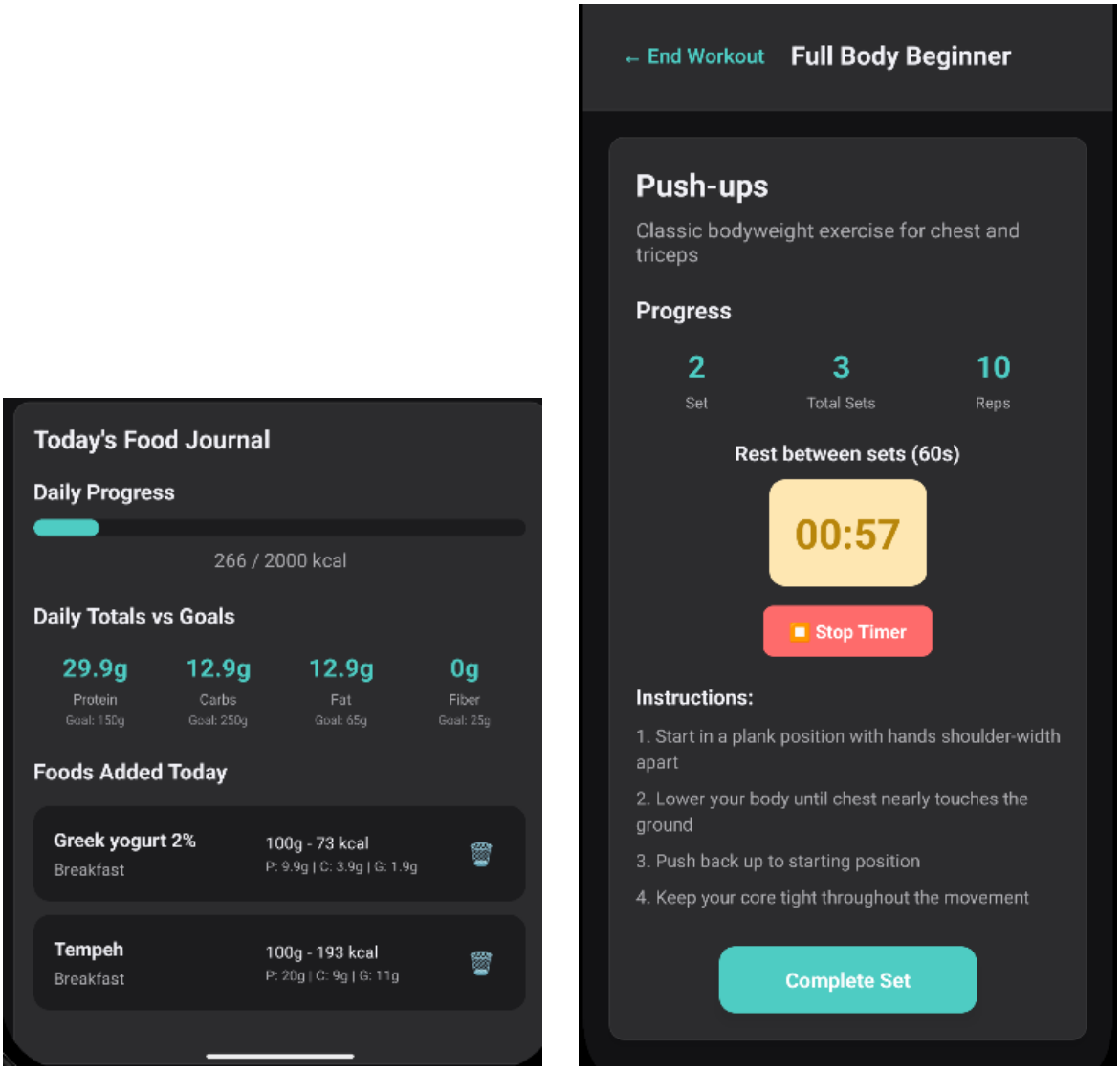

Figure 4.1: Implemented nutrition tracking and workout exercise management features.

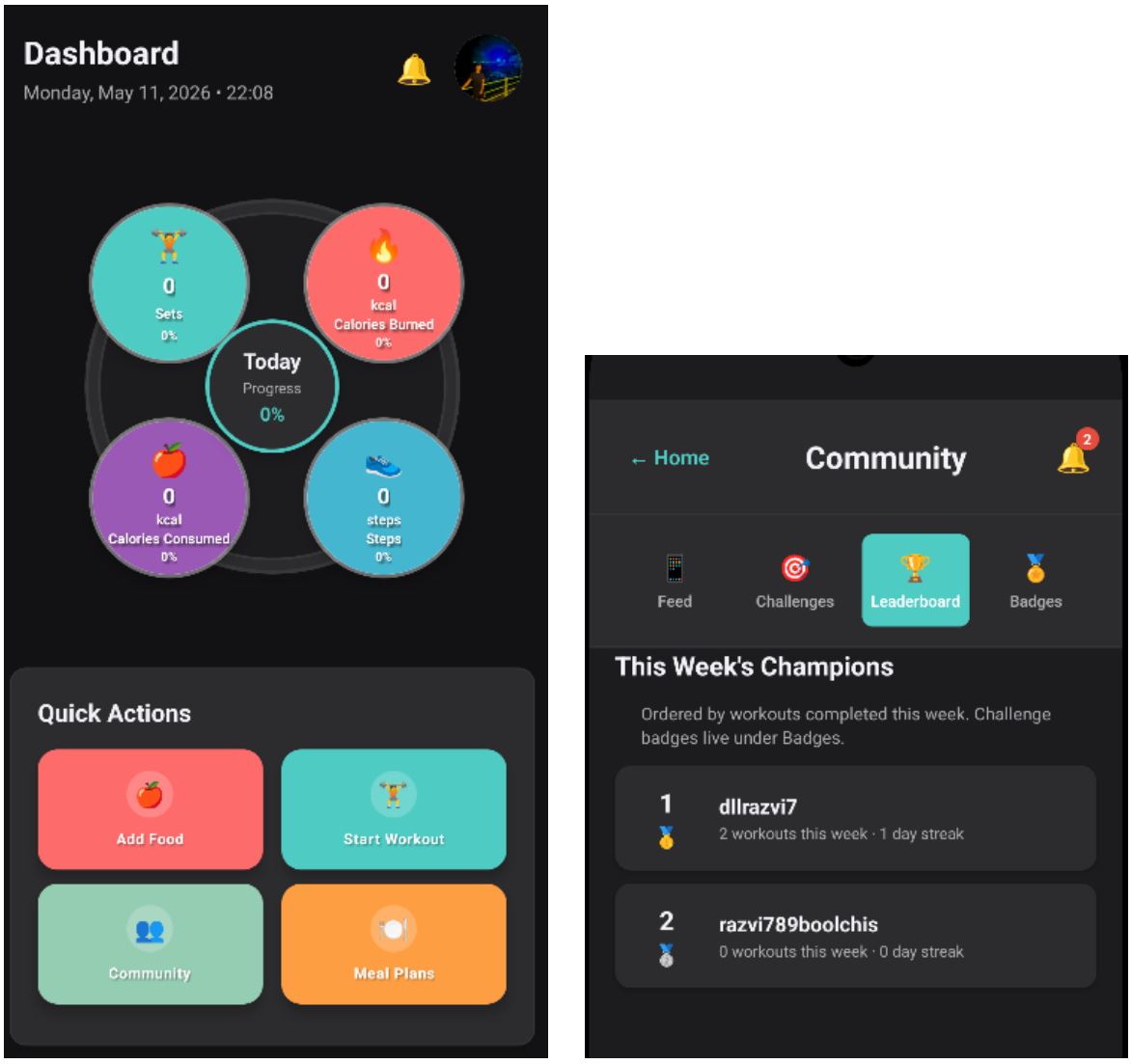

Figure 4.2: Dashboard overview and community leaderboard functionality.

Module	**Core Features Implemented**	**Status and Notes**
Nutrition	Search, filter, sort, detail, journal, goals, meal plans	Complete; hybrid catalog strategy with room for broader external databases
Workout	External catalog, body map, routines, guided sessions, planner	Complete; performance-conscious list/detail split; cloud routine snapshots
Dashboard	Concentric indicators, quick actions, context sync, steps	Complete; Google Fit step integration on supported devices
Profile	Attributes, preferences, stats, export, theme	Complete; Firestore persistence with fallback handling
Community	Feed, challenges, leaderboard, badges, notifications	Complete; real-time Firestore listeners operational

Table 4.1: Module implementation status summary

4.2 Testing and Validation

Testing was performed throughout development to verify both individual features and interactions between application modules. Because FitTrack relies on user interaction, cloud synchronization, and real-time updates, validation focused primarily on practical usage scenarios.

Main testing areas:

- *Authentication and Sessions* - login, registration, session persistence, protected actions, and error handling.
- *Nutrition* - food search, filtering, meal logging, daily goal calculations, and date consistency for nutrition journals.
- *Workout* - exercise discovery, routine creation, guided workout execution, timers, weekly planning, and workout history.
- *Dashboard* - real-time updates, progress indicators, Google Fit integration, and synchronization with nutrition and workout data.
- *Profile* - data persistence, statistics generation, user preferences, and export functionality.
- *Community* - challenge participation, leaderboard updates, notifications, likes, comments, and badge progression.

Special attention was given to cross-module interactions. Nutrition logs, workout sessions, and profile statistics were verified to ensure that updates propagated correctly across the dashboard and community features. Additional validation focused on Firestore synchronization, date handling, and recovery from temporary connectivity issues.

Testing was performed primarily on Android devices and emulators throughout development. Regression testing followed major changes to Firebase integration, workout management, nutrition tracking, and shared application contexts to ensure that new functionality did not affect existing features.

4.3 Performance Evaluation

Performance evaluation focused on responsiveness and stability during everyday application usage rather than formal benchmarking. The objective was to ensure that common workflows remained smooth while handling cloud synchronization, exercise catalogs, and frequent user interactions.

Key observations during testing:

- *Exercise catalog performance:* Rendering instructional videos directly inside exercise lists caused instability and increased resource usage. Loading media only in the exercise details screen significantly improved scrolling performance.
- *Firestore synchronization:* Nutrition logs, workout sessions, and profile updates were generally reflected across the application within a short period of time under normal network conditions.
- *Application startup behavior:* Additional safeguards were required to prevent local state from overwriting cloud data before synchronization completed.
- *Community updates:* Leaderboard rankings, challenge progress, and notifications remained responsive after improvements to synchronization logic and data propagation.
- *Step tracking integration:* Performance depended primarily on Google Fit permissions and service availability rather than application-side processing.

Although no dedicated benchmarking tools or memory profilers were used, testing on Android devices and emulators indicated stable performance during routine application usage, including nutrition tracking, workout execution, dashboard updates, and community interactions.

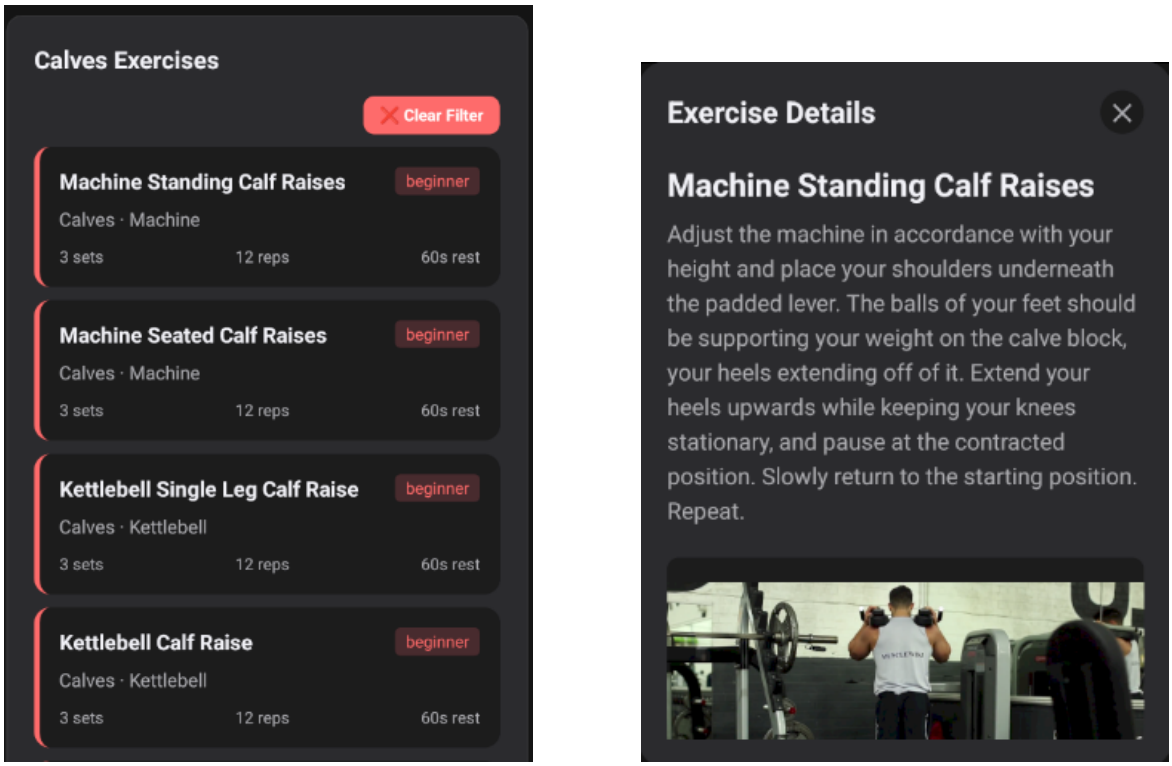

Figure 4.3: Exercise catalog view and detailed exercise screen. Instructional media is loaded only when an exercise is opened, improving scrolling performance and overall application stability.

4.4 Key Challenges and Solutions

Several challenges emerged during development, particularly in areas where multiple modules interacted with shared data. Most issues were not caused by individual screens, but by synchronization, persistence, and consistency across different parts of the application.

Date and time consistency. One recurring challenge involved handling dates across modules. Nutrition logs and workout history relied on day-based grouping, but some operations used local dates while others relied on UTC-based timestamps. This occasionally caused entries created near midnight to appear under the wrong day. The issue was resolved by adopting a consistent local date representation throughout the application.

Data synchronization and persistence. Several issues were related to Firestore synchronization. During development, nutrition journals could occasionally appear empty after authentication because local state was written to the database before existing cloud data had finished loading. Similar challenges appeared in profile management, where optimistic updates could display success messages even when cloud synchronization temporarily failed. Additional loading safeguards and validation checks were introduced to improve reliability.

Cross-module consistency. As new features were added, maintaining a single source of truth became increasingly important. Meal plans, nutrition goals, dashboard indicators, and community challenges originally relied on different data sources, which occasionally produced inconsistent results. For example, users could meet their visible protein targets while challenge progress remained unchanged. These issues were addressed by aligning the underlying data structures and synchronization logic used across modules.

Performance and usability improvements. The workout module presented several practical challenges. Early versions attempted to render instructional media directly within exercise lists, leading to instability and poor performance on Android devices. The solution was to load videos only when an exercise detail screen is opened. Additional usability improvements included redesigning routine selection workflows and refining exercise filtering logic to produce more accurate results.

Cloud integration and permissions. Community features depended heavily on Firestore collections, security rules, and real-time synchronization. During development, incomplete rule configurations occasionally prevented access to challenges, badges, or leaderboard data. Improved error handling, clearer user feedback, and revised Firestore rules helped reduce these issues and improve overall stability.

Addressing these challenges significantly improved the reliability of the application and highlighted the importance of consistent data management when multiple

modules share common information and user progress data.

4.5 Current Limitations and Future Enhancements

Although the current version of FitTrack implements the main functionality planned for the project, several limitations remain.

Current limitations:

- The application depends on external services such as Firebase, Google Fit, and exercise catalog providers. Limited connectivity or unavailable services can affect certain features.
- Most testing was performed on Android devices and emulators. While the application was designed as a cross-platform solution, validation on iOS devices remains more limited.
- Some functionality still relies on multiple synchronized data sources. During development, features such as meal plans, nutrition goals, and challenge progress required additional synchronization logic to maintain consistency.
- The workout catalog currently uses a limited retrieval strategy focused on responsiveness rather than complete repository browsing. Full search and pagination have not yet been implemented.
- The application does not currently provide a dedicated offline mode. Basic Firestore caching is available, but conflict resolution and explicit synchronization feedback are not yet supported.

Future enhancements:

- Improved exercise discovery through full-catalog search, pagination, and more advanced routine editing tools.
- Integration with additional wearable data sources, including heart rate, recovery metrics, sleep information, and hydration-related reminders, represents a common direction in modern mobile health systems [JZ16, ZDP+14].
- Expanded nutrition support through larger food databases and more flexible meal-planning functionality.
- Social sharing features that allow workout summaries and achievements to be exported to external platforms.

- Optional camera-based exercise assistance using pose estimation techniques to provide feedback on exercise execution, similar to recent trends in personalized and context-aware digital health interventions [LGKM20, NLZ+24].
- A dedicated offline mode with local storage, synchronization management, and improved support for unstable network conditions.

These limitations do not prevent normal use of the application, but they identify areas where future development could improve usability, reliability, and personalization.

4.6 Chapter Summary

This chapter evaluated the results of the FitTrack implementation and demonstrated how the architectural and design decisions presented in previous chapters translated into a functional mobile application. The completed system integrates nutrition tracking, workout management, progress monitoring, profile customization, and community features within a unified user experience.

Testing and validation confirmed the correct operation of the main user workflows, while performance evaluation highlighted the importance of efficient data loading, synchronization, and responsive mobile interfaces. Several practical challenges were encountered throughout development, particularly in areas involving cloud synchronization, cross-module consistency, and external service integration. Addressing these issues improved the overall stability and reliability of the application.

Although some limitations remain, the current implementation successfully achieves the primary objectives defined at the beginning of the project and provides a solid foundation for future enhancements. The next chapter reflects on the development process, the lessons learned, and potential directions for the continued evolution of FitTrack.

Chapter 5

Conclusions and Future Work

This chapter reflects on the development of FitTrack, summarizing the main achievements of the project, the lessons learned throughout implementation, and the directions that could guide future development. While Chapter 4 focused on evaluation and testing results, this chapter takes a broader view of the project and its overall outcomes.

5.1 Project Summary and Achievements

FitTrack demonstrates the feasibility of combining fitness tracking, nutrition management, and social engagement within a single mobile application. Developed using *React Native*, *TypeScript*, and *Firebase*, the project brings together several functionalities that are often distributed across separate platforms.

From a user perspective, the application supports nutrition logging, meal planning, workout tracking, personal routine management, progress monitoring, and community interaction. Users can search and record meals, create workout routines, follow guided training sessions, monitor daily activity through the dashboard, manage personal goals, and participate in community challenges and leaderboards.

A significant achievement of the project is the integration of these features into a unified workflow. Nutrition data, workout activity, dashboard statistics, profile information, and community features are connected through shared application state and cloud-based persistence, allowing updates in one area of the application to be reflected across others.

From a technical perspective, the project resulted in a modular architecture built around reusable components, shared context providers, and Firestore-backed persistence. Particular attention was given to workout management, where exercise routines are stored together with exercise snapshots to ensure reliable execution even when external catalog data changes. The integration of external exercise providers,

together with performance-oriented loading strategies, allowed access to rich exercise content while maintaining responsive mobile interactions.

Overall, the project achieved the primary objectives established at the beginning of the thesis and resulted in a functional prototype capable of supporting everyday fitness and nutrition tracking workflows.

5.2 Insights and Lessons Learned

Developing FitTrack provided valuable experience not only with mobile technologies, but also with the practical challenges of integrating multiple features into a single application. Many of the most important lessons did not come from implementing individual screens, but from ensuring that data remained consistent across nutrition, workout, profile, dashboard, and community modules.

One recurring lesson was the importance of establishing a single source of truth for shared data. Several issues encountered during development were caused by different modules relying on separate representations of the same information. Examples included nutrition goals used by meal plans, challenge progress calculations, and dashboard statistics. Resolving these inconsistencies required careful synchronization and highlighted the importance of designing data flows early in the development process.

Another important lesson involved balancing functionality and performance. Early versions of the workout catalog attempted to display instructional videos directly within exercise lists. Although this provided immediate access to media content, it negatively affected performance and stability. Moving videos to dedicated detail screens resulted in a significantly better user experience while preserving access to the same information.

The project also reinforced the importance of handling dates and cloud synchronization carefully. Issues involving local time versus UTC timestamps, delayed Firestore synchronization, and startup data hydration demonstrated that seemingly simple features can become complex when data is shared across multiple screens and persisted remotely.

Some of the most important lessons learned during development were:

- Prioritize data consistency before adding new features.
- Keep mobile interfaces lightweight and load heavy content only when necessary.
- Design for synchronization and offline scenarios from the beginning.

- Treat cross-module integration as carefully as individual feature implementation.
- Favor simple and predictable user flows over feature-rich but complex interactions.

5.3 Future Development Directions

Although the current version of FitTrack provides a complete fitness and nutrition tracking workflow, the project was designed with future expansion in mind. The modular architecture and cloud-based data model allow new functionality to be added incrementally without requiring major structural changes.

Near-term priorities:

- Full exercise search and pagination across external exercise catalogs.
- More advanced routine management, including exercise reordering, streamlined editing, and improved workout composition tools.
- A dedicated offline mode with local persistence, queued synchronization, and conflict resolution.
- Expanded nutrition databases together with optional barcode scanning for faster food logging.

Longer-term opportunities:

- *Wearables and recovery:* integration with additional health data sources such as heart rate, sleep quality, recovery indicators, and hydration-related reminders.
- *Computer vision support:* camera-assisted exercise analysis capable of providing basic feedback on movement quality and exercise execution.
- *Social sharing:* exportable workout summaries and achievement cards that can be shared outside the application while preserving user privacy controls.
- *Nutrition recommendations:* connecting remaining daily macros to practical meal suggestions, food discovery tools, or local purchasing options.
- *Personalization:*More advanced recommendation systems for training and nutrition based on user behavior and historical activity patterns, following approaches explored in personalized fitness and nutrition recommendation research [LGKM20, YHY+17].

Many of these extensions build directly on functionality already present in the application. As a result, future development can focus on expanding existing workflows rather than redesigning the underlying architecture.

5.4 Reflections on Mobile Health Development

Developing FitTrack highlighted both the opportunities and challenges involved in creating modern mobile health applications. From a technical perspective, current frameworks and cloud services make it possible for small teams or individual developers to build feature-rich systems that would have required significantly more resources in the past.

At the same time, the project demonstrated that building a health-related application involves more than implementing individual features. Consistency between modules, reliable data synchronization, intuitive user interfaces, and clear presentation of information are often just as important as the underlying functionality itself. Small issues involving dates, synchronization, or user workflows can have a noticeable impact on the overall user experience.

Another important observation is that users increasingly expect fitness, nutrition, and social features to work together rather than as isolated tools. Research on gamification and digital behavior change suggests that combining activity tracking, goal setting, and social engagement mechanisms can improve long-term participation in health-related applications [HKS14, SASL17].. Bringing these elements into a single application creates opportunities for a more complete view of personal progress, but also introduces additional complexity in terms of data management and system integration.

FitTrack represents one possible approach to this challenge. While the application remains a prototype and several improvements are still possible, the project demonstrates that an integrated fitness and nutrition platform can be built using modern cross-platform technologies while maintaining a coherent user experience and a scalable architectural foundation.

Chapter 6

Conclusions

This chapter presents the final conclusions of the FitTrack project. It revisits the objectives established at the beginning of the thesis, summarizes the main contributions of the implementation, and reflects on the overall outcome of the work.

6.1 Fulfillment of Objectives

The primary objective of this project was to design and implement a mobile application capable of combining fitness tracking, nutrition management, and user engagement features within a single platform. This objective was successfully achieved through the development of FitTrack, a cross-platform application built with React Native, TypeScript, and Firebase.

The resulting system supports nutrition logging, workout planning, daily progress monitoring, profile management, and community interaction through an integrated user experience. Rather than treating these features as independent modules, the application connects them through shared data structures and real-time synchronization, allowing information entered in one area of the system to be reflected throughout the rest of the application.

In addition to delivering the planned functionality, the project demonstrated the feasibility of building a modern mobile health application without requiring dedicated server infrastructure. Firebase services provided authentication, cloud storage, and synchronization capabilities, while React Native enabled the development of a shared codebase targeting multiple mobile platforms.

6.2 Main Contributions

The development of FitTrack resulted in several practical and technical contributions. The project demonstrates how fitness tracking, nutrition management, and community engagement can be integrated into a single mobile application while maintaining a consistent user experience.

A key contribution is the modular architecture built around React Native, Firebase, and shared context providers, allowing multiple application modules to remain synchronized through a common data layer. Another contribution is the workout management workflow, where exercise routines are stored together with exercise snapshots, ensuring reliable session execution even when external catalog data changes.

The project also highlights the importance of performance-oriented design decisions in mobile applications. Examples include separating lightweight catalog browsing from media-rich exercise details, using virtualized lists for large datasets, and maintaining responsive interfaces through selective data loading and real-time synchronization.

Together, these contributions provide a practical foundation for future development while demonstrating the feasibility of building an integrated mobile health platform using modern cross-platform technologies.

6.3 Final Conclusions

This thesis presented the design and implementation of FitTrack, a mobile application that combines nutrition tracking, workout management, progress monitoring, and community interaction within a unified platform. The project evolved from an initial concept into a functional prototype capable of supporting everyday fitness and wellness activities.

Throughout development, the project demonstrated that building an effective health-related application involves much more than implementing individual features. Consistent data management, reliable synchronization, responsive interfaces, and intuitive user workflows proved equally important to the overall quality of the system. Many of the challenges encountered during implementation emerged at the boundaries between modules rather than inside individual features, reinforcing the importance of integration-focused design.

The completed application successfully fulfills the objectives established at the beginning of the project and provides a solid foundation for future en-

hancements. While additional functionality can still be added, the current implementation demonstrates the viability of an integrated approach to fitness and nutrition management and offers valuable insight into the development of modern mobile health applications.

Bibliography

[HKS14] Juho Hamari, Jonna Koivisto, and Harri Sarsa. Does gamification work? a literature review of empirical studies on gamification. In *Proceedings of the 47th Hawaii International Conference on System Sciences*, pages 3025–3034. IEEE, 2014.

[JZ16] Ting Jiang and Yong J. Yuan Zhang. Wearable medical monitoring systems based on wireless networks: A review. *IEEE Sensors Journal*, 16(23):8186–8199, 2016.

[LGKM20] Peng Liao, Kristjan Greenewald, Predrag Klasnja, and Susan Murphy. Personalized heartsteps: A reinforcement learning algorithm for optimizing physical activity. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 4(1):1–22, 2020.

[NLZ+24] Aila Naderbagi, Victoria Loblay, IUM Zahed, Mahalakshmi Ekambareshwar, Adam Poulsen, Yun J C Song, Laura Ospina-Pinillos, Michael Krausz, Mostafa Mamdouh Kamel, Ian B. Hickle, and Haley M. LaMonica. Cultural and contextual adaptation of digital health interventions: Narrative review. *Journal of Medical Internet Research*, 26:e55130, 2024.

[PAL20] Philips Kokoh Prasetyo, Palakorn Achananuparp, and Ee-Peng Lim. Foodbot: A goal-oriented just-in-time healthy eating interventions chatbot. In *Proceedings of the International Conference on Digital Health*, pages 79–88, 2020.

[PAV15] Mitesh S. Patel, David A. Asch, and Kevin G. Volpp. Wearable devices as facilitators, not drivers, of health behavior change. *Journal of the American Medical Association*, 313(5):459–460, 2015.

[SASL17] Ali Shameli, Tim Althoff, Amin Saberi, and Jure Leskovec. How gamification affects physical activity: Large-scale analysis of walking challenges in a mobile application. *Proceedings of the International AAAI Conference on Web and Social Media*, 11(1), 2017.

[SGL+21] Md. Shamsujjoha, John Grundy, Li Li, Hourieh Khalajzadeh, and Qinghua Lu. Human-centric issues in ehealth app development and usage: A preliminary assessment. In *IEEE International Conference on Software Analysis, Evolution and Reengineering*, pages 752–756, 2021.

[SHKW15] Stoyan R. Stoyanov, Leanne Hides, David J. Kavanagh, and Helen Wilson. Mobile app rating scale: A new tool for assessing the quality of health mobile apps. *JMIR mHealth and uHealth*, 3(1):e27, 2015.

[SP13] Akane Sano and Rosalind W Picard. Stress recognition using wearable sensors and mobile phones. In *2013 Humaine Association Conference on Affective Computing and Intelligent Interaction*, pages 671–676. IEEE, 2013.

[Str25] Strava Inc. Strava. `https://www.strava.com`, 2025. Accessed May 2025.

[SVS+17] Brendon Stubbs, Davy Vancampfort, Lee Smith, Simon Rosenbaum, Felipe B. Schuch, and Joseph Firth. Exercise as a treatment for depression: A meta-analysis adjusting for publication bias. *Journal of Psychiatric Research*, 77:42–51, 2017.

[THP15] Yi-Yuan Tang, Britta K. Hölzel, and Michael I. Posner. The neuroscience of mindfulness meditation. *Nature Reviews Neuroscience*, 16(4):213–225, 2015.

[WHK05] Pornratshanee Weerapong, Patria A Hume, and Gregory S Kolt. The mechanisms of massage and effects on performance, muscle recovery and injury prevention. *Sports Medicine*, 35:235–256, 2005.

[YHY+17] Longqi Yang, Cheng-Kang Hsieh, Hongjian Yang, John Pollak, Nicola Dell, Serge Belongie, Curtis Cole, and Deborah Estrin. Yum-me: A personalized nutrient-based meal recommender system. In *Proceedings of the ACM Conference on Recommender Systems*, pages 37–45, 2017.

[ZDP+14] Ya-Li Zheng, Xiao-Rong Ding, Carmen Chung Yan Poon, Benny Ping Lai Lo, Heye Zhang, Xiao-Lin Zhou, Guang-Zhong Yang, Ni Zhao, and Yuan-Ting Zhang. Unobtrusive sensing and wearable devices for health informatics. *IEEE Transactions on Biomedical Engineering*, 61(5):1538–1554, 2014.